

L89 R2.0&LC29H&LC79H Series Firmware Upgrade Guide

GNSS Products

Version: 1.8

Date: 2025-08-05

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2025. All rights reserved.

About the Document

Document Information	
Title	L89 R2.0&LC29H&LC79H Series Firmware Upgrade Guide
Subtitle	GNSS Products
Document Type	Firmware Upgrade Guide
Document Status	Released

Revision History

Version	Date	Description
-	2021-04-28	Creation of the document
1.0	2021-05-19	First official release
1.1	2021-08-18	Added the applicable module LC79H.
1.2	2022-04-07	- Added the default UART baud rate for firmware upgrades of the applicable modules. - Updated List of Constants and added a note on the format length for LC29H (BA, CA) (Table 3). - Added the information about DA files and updated the checksum code of DA file (Chapter 2.2.3). - Updated the diagram of Synchronize with DA and Get DA Report (Figure 13). - Updated the diagram of Format Flash (Figure 15). - Updated the diagram of Send FW File to Module, added checksum code of the FW packet, and added notes about the FW file and FW packet (Chapter 2.2.7). - Updated the upgrade implementation example (Chapter 2.3). - Deleted the chapter of Disconnect Module.
1.3	2022-07-15	Added the table of applicable modules and removed the note on the format length for LC29H (BA, CA) (Table 1).

Version	Date	Description
		- Added the table of format length and FW flash address based on software versions of applicable modules (Table 6). - Updated the flow chart of communication between host and module (Figure 3). - Added the timeout period for the module to respond to the host command (Chapter 2.2.1). - Updated the upgrade implementation example (Chapter 2.3).
1.4	2023-01-13	- Updated applicable variants: Added applicable variant LC29H (BS). Updated applicable variant LC79H Series to LC79H (AL). - Added the firmware upgrading via I2C interface. - Added a note about firmware download by PC's software utility (Chapter 1). - Updated List of Constants and added a note on the format length for LC29H (BA, CA) (Table 3).
1.5	2023-06-28	- Added information on firmware upgrade in moving base application (Chapters 1.2 and 2.1). - Updated FW flash address and added a note about FW flash address for general application (Chapter 2.1).
1.6	2024-07-27	- Added applicable modules/devices L89 (HD), LC29H (AI), and QLM29H series. - Added selecting the interface for firmware upgrade via hardware pins in Download mode (Chapter 2.2). - Added information on Upgrade mode (Chapter 2.3). - Added examples of firmware upgrade in Upgrade mode via SPI and I2C (Chapters 3.3 and 3.4). - Added firmware upgrade example by QGNSS (Chapter 4).
1.7	2025-03-11	- Deleted L89 (HB), LC29H (BS) and QLM29H series according to Quectel's product marketing strategies. - Deleted Flash Manufacturer ID, Flash Device ID 1 and Flash Device ID 2 (Table 3). - Added the list of SPI status flags in Upgrade mode (Table 5). - Added module flash parameters (Table 8). - Added a note about timeout operation for receiving formatting progress when formatting flash via UART in Download mode (Chapter 2.2.6.1). - Added a note about the maximum data size that the I2C interface can receive at one time (Chapter 2.2.7.2). - Added notes about the transmitting format of module status, firmware address, firmware information parameters and format flash frame parameters in Upgrade mode (Chapters 2.3.2, 2.3.3 and 2.3.5). - Updated the example for firmware upgrade via SPI in Upgrade mode (Chapter 3.3).

Version	Date	Description
		9. Updated the versions of QGNSS tool and GNSS MODULE EVB used in firmware upgrade (Chapter 4).
1.8	2025-08-05	Added applicable module LC79H (EL).

Contents

About the Document.....	3
Contents	6
Table Index.....	7
Figure Index	8
1 Introduction	10
1.1. General Applications.....	11
1.2. Moving Base Application.....	11
2 Firmware Upgrade Process	13
2.1. Firmware Information	17
2.2. Download Mode	18
2.2.1. Handshake with Module.....	20
2.2.2. Disable Module WDT	22
2.2.3. Send DA File to Module	23
2.2.4. Jump to DA.....	26
2.2.5. Synchronize with DA	27
2.2.6. Format Flash	29
2.2.6.1. Format via UART	29
2.2.6.2. Format via I2C.....	30
2.2.7. Send FW File to Module	32
2.2.7.1. Send FW File via UART	33
2.2.7.2. Send FW File via I2C	34
2.3. Upgrade Mode	37
2.3.1. Handshake with Module.....	40
2.3.2. Send Firmware Address to Module	40
2.3.3. Send Firmware Information to Module.....	43
2.3.4. Send Start Firmware Upgrade Command to Module	44
2.3.5. Send Format Flash Message to Module.....	45
2.3.6. Send FW File to Module	47
3 Upgrade Implementation Examples.....	49
3.1. Download Mode – Firmware Upgrade via UART Interface	49
3.2. Download Mode – Firmware Upgrade via I2C Interface	56
3.3. Upgrade Mode – Firmware Upgrade via SPI.....	67
3.4. Upgrade Mode – Firmware Upgrade via I2C Interface.....	82
4 Upgrade Example by QGNSS	89
4.1. Procedure in Download Mode for General Applications.....	89
4.2. Procedure in Download Mode for Moving Base Application	96
5 Appendix References	98

Table Index

Table 1: Applicable Modules	10
Table 2: Download and Upgrade Modes	13
Table 3: List of Constants in Download Mode	13
Table 4: List of Constants in Upgrade Mode	15
Table 5: List of SPI Status Flags in Upgrade Mode.....	16
Table 6: Format Length and FW Flash Address (General Applications).....	17
Table 7: Format Length and FW Flash Address (Moving Base Application)	17
Table 8: Flash Parameters	28
Table 9: Description of Binary Protocol Fields.....	30
Table 10: RACE_DA_FORMAT_FLASH	30
Table 11: RACE_DA_FORMAT_FLASH Payload	31
Table 12: RACE_DA_FORMAT_FLASH_RESPONSE	31
Table 13: RACE_DA_FORMAT_FLASH_RESPONSE Payload	31
Table 14: RACE_DA_WRITE_DATA.....	34
Table 15: RACE_DA_WRITE_DATA Payload.....	34
Table 16: RACE_DA_WRITE_DATA_RESPONSE.....	34
Table 17: RACE_DA_WRITE_DATA_RESPONSE Payload	34
Table 18: Protocol Structure in Upgrade Mode	41
Table 19: ClassID and MsgID	41
Table 20: Module Status in Upgrade Mode	42
Table 21: Firmware Address Frame from Host.....	42
Table 22: Firmware Address Response Frame from Module.....	43
Table 23: Firmware Information Frame from Host.....	43
Table 24: Firmware Information Parameters	43
Table 25: Firmware Information Response Frame from Module	44
Table 26: Start Firmware Upgrade Frame from Host	45
Table 27: Start Firmware Upgrade Response Frame from Module	45
Table 28: Format Flash Frame from Host.....	46
Table 29: Format Flash Frame Parameters.....	46
Table 30: Format Progress Frame from Module.....	46
Table 31: Format Progress Response Frame from Host.....	46
Table 32: Format Flash Response Frame from Module.....	47
Table 33: FW File Frame from Host.....	47
Table 34: FW File Response Frame from Module.....	48
Table 35: Related Documents.....	98
Table 36: Terms and Abbreviations	98

Figure Index

Figure 1: Firmware Upgrade Connection – UART/I2C/SPI	11
Figure 2: Schematic Diagram of Connection Between Base and Rover.....	12
Figure 3: Communication Process in Download Mode.....	18
Figure 4: Data Received via I2C	19
Figure 5: Handshake – UART	20
Figure 6: Handshake – I2C	21
Figure 7: Disable Module WDT – UART	22
Figure 8: Disable Module WDT – I2C	23
Figure 9: Send DA File to Module – UART	24
Figure 10: Send DA File to Module – I2C	25
Figure 11: Jump to DA – UART	26
Figure 12: Jump to DA – I2C.....	26
Figure 13: Synchronize with DA and Get DA Report – UART.....	27
Figure 14: Synchronize with DA and Get DA Report – I2C.....	28
Figure 15: Format Flash – UART	29
Figure 16: Binary Protocol Structure.....	30
Figure 17: Format Flash – I2C	32
Figure 18: Send FW File to Module – UART	33
Figure 19: Send FW File to Module – I2C.....	35
Figure 20: Communication Process in Upgrade Mode.....	37
Figure 21: Module SPI Read/Write Flow.....	38
Figure 22: Module I2C Read/Write Flow.....	39
Figure 23: Handshake.....	40
Figure 24: Protocol Structure in Upgrade Mode	41
Figure 25: Send Firmware Address	42
Figure 26: Send Firmware Information	43
Figure 27: Send Start Firmware Upgrade Command	44
Figure 28: Format Flash.....	45
Figure 29: Send FW Flow	47
Figure 30: EVB Connection.....	89
Figure 31: Device Manager.....	90
Figure 32: Device Information.....	91
Figure 33: Firmware Download	91
Figure 34: Select Download Baud Rate.....	92
Figure 35: Load Firmware	92
Figure 36: Firmware Selection	92
Figure 37: Wait for Handshake	93
Figure 38: Handshake Succeeded.....	94
Figure 39: Wait for Upgrading to Complete	95
Figure 40: Firmware Upgrade Completed.....	96
Figure 41: Firmware Selection of LC29H (EA) in Moving Base Application.....	97

Figure 42: Upgrade Completed.....	97
-----------------------------------	----

1 Introduction

This document outlines the procedure for upgrading the firmware of the Quectel L89 R2.0, LC29H series and LC79H series GNSS modules.

Table 1: Applicable Modules

Module Series	Model
L89 R2.0	L89 (HA)
	L89 (HD)
LC29H	LC29H (AA)
	LC29H (AI)
	LC29H (BA)
	LC29H (CA)
	LC29H (DA)
	LC29H (EA)
LC79H	LC79H (AL)
	LC79H (EL)

NOTE

If the firmware upgrade fails or the process is stopped by the user, no backup firmware can be executed. The only way to reprogram a non-working firmware is to reset the hardware and start a new upgrade process.

1.1. General Applications

Following the procedure illustrated in this document, you can upgrade the firmware to the target GNSS module via UART, I2C, or SPI. When used for firmware upgrade, the default baud rate of UART interface is 115200 bps, the maximum baud rate of I2C interface is 400 kbps, and the maximum clock frequency of SPI is 48 MHz.

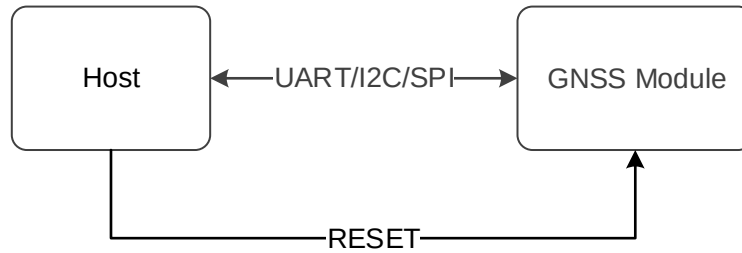


Figure 1: Firmware Upgrade Connection – UART/I2C/SPI

NOTE

For the interfaces supported by a specific GNSS module model, refer to the corresponding hardware design or user guide.

1.2. Moving Base Application

In the LC29H (EA) dual modules, one can be selected as a base and the other as a rover, to achieve moving base application. The rover communicates with the UART2 on base via UART1 and you can download the software files via UART1 on base. For details about the connection between two modules, see [Figure 2: Schematic Diagram of Connection Between Base and Rover](#).

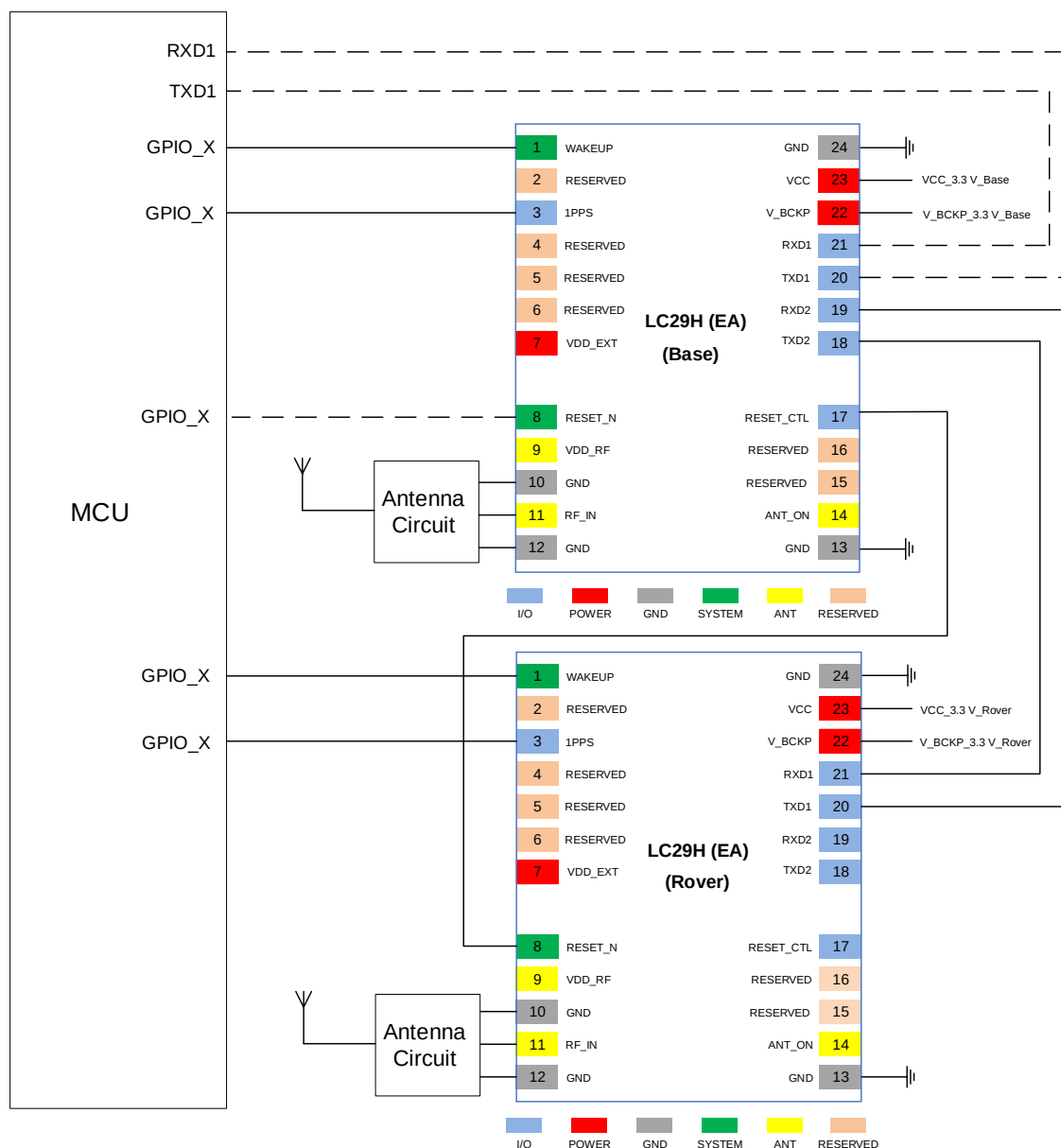


Figure 2: Schematic Diagram of Connection Between Base and Rover

2 Firmware Upgrade Process

The GNSS module supports two firmware upgrade modes: Download and Upgrade. The main differences between the two modes are listed below.

Table 2: Download and Upgrade Modes

Firmware Upgrade Mode	Supported Interface	Supported Module
Download	UART	L89 R2.0, LC29H series and LC79H series
	I2C	LC29H (AA, AI, BA, CA, DA)
Upgrade	SPI	LC29H (AI)
	I2C	LC29H (AI) and LC79H (AL)

For details on the constants in different modes, see [Table 3: List of Constants in Download Mode](#) and [Table 4: List of Constants in Upgrade Mode](#).

This chapter describes all the necessary steps in the firmware upgrade process.

Table 3: List of Constants in Download Mode

Constant Name	Constant Value	Steps
Communication Address (I2C)	0001 0000 (W bit)	Chapter 2.2 Download Mode
	0001 0001 (R bit)	Chapter 2.2 Download Mode
I2C Data Mark ¹⁾	0x66	Chapter 2.2 Download Mode
HW_CODE	0x3335	Figure 6: Handshake – I2C
BROM_ERROR	0x1000	Figure 6: Handshake – I2C Figure 7: Disable Module WDT – UART

Constant Name	Constant Value	Steps
		Figure 8: Disable Module WDT – I2C Figure 9: Send DA File to Module – UART Figure 10: Send DA File to Module – I2C Figure 11: Jump to DA – UART Figure 12: Jump to DA – I2C
Module WDT Register Address	0xA2080000	Figure 7: Disable Module WDT – UART
WDT Value	0x0010	Figure 8: Disable Module WDT – I2C
	0x04204000 (UART)	Figure 9: Send DA File to Module – UART Figure 11: Jump to DA – UART
DA Run/Start Address	0x04000000 (I2C)	Figure 10: Send DA File to Module – I2C Figure 12: Jump to DA – I2C
DA Length ²⁾	0x00006CF7 (DA file requiring 115200 bps for transmission via UART) 0x00006DCB (DA file requiring 921600 bps for transmission via UART)	Figure 9: Send DA File to Module – UART
	0x00006A44 (DA file transmission via I2C)	Figure 10: Send DA File to Module – I2C
Flash Mount Status	0x00000000	Figure 13: Synchronize with DA and Get DA Report – UART
Flash Start Address	0x08000000	Figure 14: Synchronize with DA and Get DA Report – I2C
Flash Size	0x00400000	
Format Physical Address	0x08000000	
Format Length	See Table 6: Format Length and FW Flash Address (General Applications) and Table 7: Format Length and FW Flash Address (Moving Base Application) .	Figure 15: Format Flash – UART

Constant Name	Constant Value	Steps
FW Flash Address	See Table 6: Format Length and FW Flash Address (General Applications) and Table 7: Format Length and FW Flash Address (Moving Base Application) .	Figure 18: Send FW File to Module – UART Figure 19: Send FW File to Module – I2C
FW Packet Length	0x00001000	

NOTE

- ¹⁾ I2C data mark ensures the validity of received data. In case of FW upgrading via I2C interface, the host will receive response includes I2C data mark.
- ²⁾ DA file size may vary. Check the DA file size before using the DA file.
- Data are transmitted in big-endian format via UART interface, and in little-endian format via I2C interface.

Table 4: List of Constants in Upgrade Mode

Interface	Constant Name	Constant Value	Description
SPI	SPIS_ADDRESS_ID	0x55AA0000	Module SPI upgrade register.
	CFG_RD_CMD	0x0A	Configures register address and data length of the data to be read.
	RD_CMD	0x81	Reads data.
	CFG_WR_CMD	0x0C	Configures register address and data length of the data to be written.
	WR_CMD	0x0E	Writes data.
	RS_CMD	0x06	Reads module SPI status flag.
	WS_CMD	0x08	Erases module SPI read/write error flag.
	PWON_CMD	0x04	Powers on module SPI.
	PWOFF_CMD	0x02	Powers off module SPI.
SPI/I2C	HANDSHAKE_WORD1	0x514C1309	Handshake data 1.
	RSP_WORD1	0xAAFC3A4D	Handshake response data 1.
	HANDSHAKE_WORD2	0x1203A504	Handshake data 2.
	RSP_WORD2	0x55FD5BA0	Handshake response data 2.

Interface	Constant Name	Constant Value	Description
I2C	I2C_DATA_LEN_REG	0xAA510008	Module I2C data length register.

Table 5: List of SPI Status Flags in Upgrade Mode

Flag Name	Bit No.	Description
STA_SLV_ON	Bit 0	Module SPI power on flag. Bit 0 = 1 indicates module SPI is powered on.
STA_CFG_SUCCESS	Bit 1	Module SPI configuration address and length success flag. This flag cannot be cleared. Bit 1 = 1 indicates SPI configures address and length successfully.
STA_TXRX_FIFO_RDY	Bit 2	Module SPI transmit/receive buffer with FIFO ready flag. This flag is cleared after the module receives CFG_RD_CMD or CFG_WR_CMD . Bit 2 = 1 indicates module SPI transmit/receive buffer FIFO is ready.
STA_RD_ERR	Bit 3	Module SPI read error flag. This flag needs to be cleared. To clear it, send WS_CMD and STA_RD_ERR (0x08) to the module. Bit 3 = 1 indicates a read error occurs in the module SPI.
STA_WR_ERR	Bit 4	Module SPI write error flag. This flag needs to be cleared. To clear it, send WS_CMD and STA_WR_ERR (0x10) to the module. Bit 4 = 1 indicates a write error occurs in the module SPI.
STA_RDWR_FINISH	Bit 5	Module SPI read/write finish flag. This flag is cleared after the module receives CFG_RD_CMD or CFG_WR_CMD . Bit 5 = 1 indicates module SPI finishes data read/write.
RESERVED	Bit 6	Reserved.
STA_CMD_ERR	Bit 7	Module SPI command error flag. This flag is cleared after the module receives a correct command. Bit 7 = 1 indicates module SPI receives a wrong command.

2.1. Firmware Information

Table 6: Format Length and FW Flash Address (General Applications)

Module	Software Version	Format Length	FW Flash Address ¹⁾
L89 (HA)	L89HANR01A05S and previous versions	0x00400000	0x08000000 (<i>partition_table.bin</i>)
LC29H (AA)	LC29HAANR01A03S and previous versions		0x08003000 (<i>ag3335_bootloader.bin</i>)
LC29H (BA)	LC29HBANR01A01S		0x08013000 (<i><Version>.bin</i> or <i>gnss_demo.bin</i>)
LC79H (AL)	LC79HALNR01A01S		0x083FF000 (<i>gnss_config.bin</i>)
Other software versions of all applicable modules (except LC29H (EA) in Moving Base application)		0x003E0000	0x08000000 (<i>partition_table.bin</i>) 0x08003000 (<i>ag3335_bootloader.bin</i> / <i>bootloader.bin/master_bootloader.bin</i>) 0x08013000 (<i><Version>.bin</i>) 0x083DF000 (<i>gnss_config.bin</i>)

Table 7: Format Length and FW Flash Address (Moving Base Application)

Module	Software Version	Format Length	FW Flash Address ²⁾
LC29H (EA) Moving Base	All versions	0x003E0000	0x08000000 (<i>partition_table.bin</i>) 0x08003000 (<i>master_bootloader.bin</i>) 0x08013000 (<i><Version>.bin</i>) 0x08230000 (<i>slave_gnss_config.bin</i>) 0x08240000 (<i>slave_bootloader.bin</i>) 0x08250000 (<i>slave_da_921600.bin</i>) 0x08260000 (<i>slave_rom_size.bin</i>) 0x083DF000 (<i>master_gnss_config.bin</i>)

NOTE

- ¹⁾ For FW Flash address of general application, please refer to the file *flash_download.cfg*.
- ²⁾ For FW Flash address of moving base application, please refer to the file *flash_download_ms.cfg*.

2.2. Download Mode

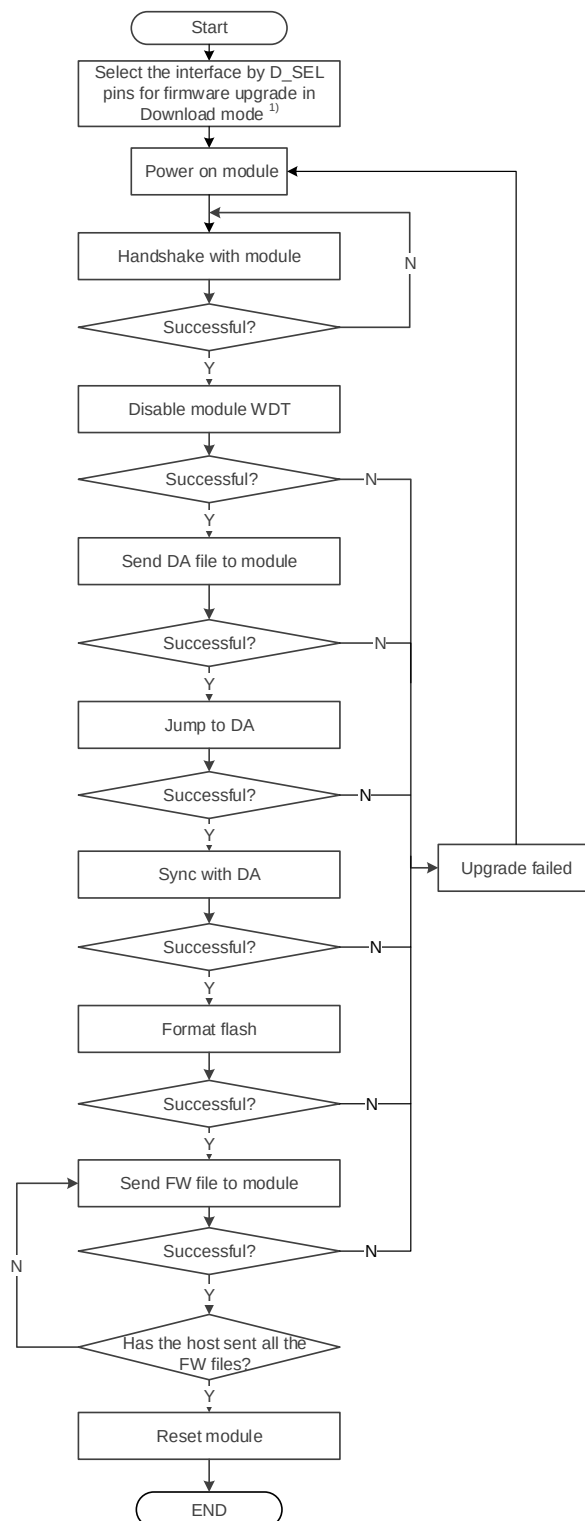


Figure 3: Communication Process in Download Mode

NOTE

¹⁾ L89 R2.0, LC29H (EA) and LC79H (AL, EL) do not support firmware upgrade in Download mode through the I2C interface, and selecting the interface is not involved in the above process. On the other hand, LC29H (AA, AI, BA, CA, DA) supports selecting the interface for firmware upgrade in Download mode:

- D_SEL1 = 0, D_SEL2 = 0: select UART1 interface (default)
- D_SEL1 = 1, D_SEL2 = 1: select I2C interface

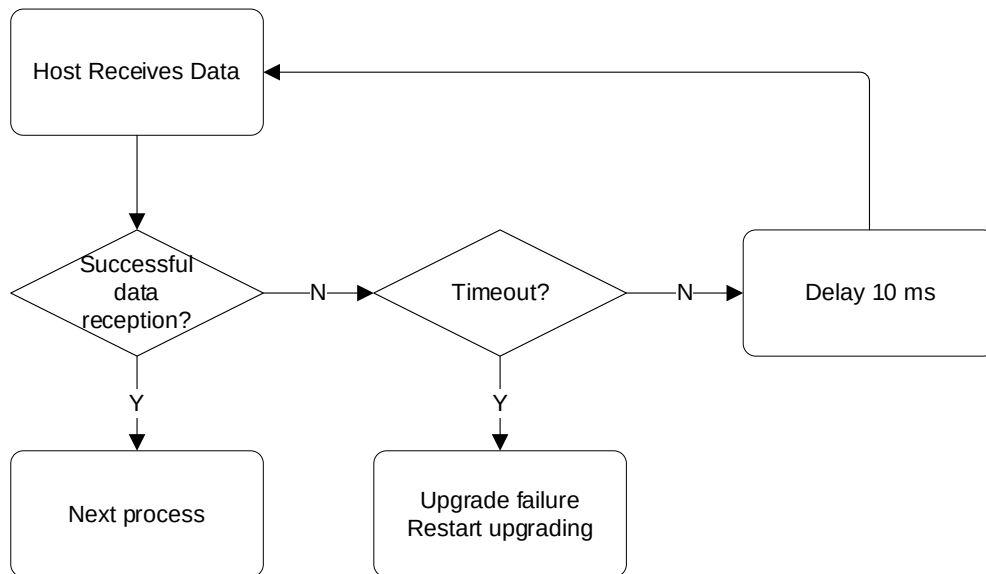


Figure 4: Data Received via I2C

The following sections illustrate how the host establishes communication with the module and sends commands and files to the module in Download mode:

- Handshake with the module
- Disable module WDT (Watch Dog Timer)
- Send DA (Download Agent) file to the module
- Jump to DA
- Synchronize with DA
- Format flash
- Send FW (firmware) file to the module

NOTE

1. As shown in [Figure 4: Data Received via I2C](#), in case of firmware upgrading via I2C interface, it is recommended to read data cyclically at 10 ms intervals up to the timeout for each reception step or execute the next step after receiving correct data. And the timeout can be set by the customer.
2. The module will output firmware version information automatically after each boot-up. For example,

LC29H (BA) will output the following message after its boot-up:

\$PQTMVERNO,LC29HBANR01A04S_CSA4,2022/09/16,15:19:05*0F

You can also execute **\$PQTMVERNO*58** to query the firmware version information. For more details, see [documents \[1\]](#), [\[2\]](#) and [\[3\] protocol specifications](#).

2.2.1. Handshake with Module

This section describes the handshaking mechanism for the communication between the host and the module.

Reset the module after it is powered on and receives **0xA0** from the host. If the module does not receive **0xA0** within 150 milliseconds after being reset, the handshake process will fail.

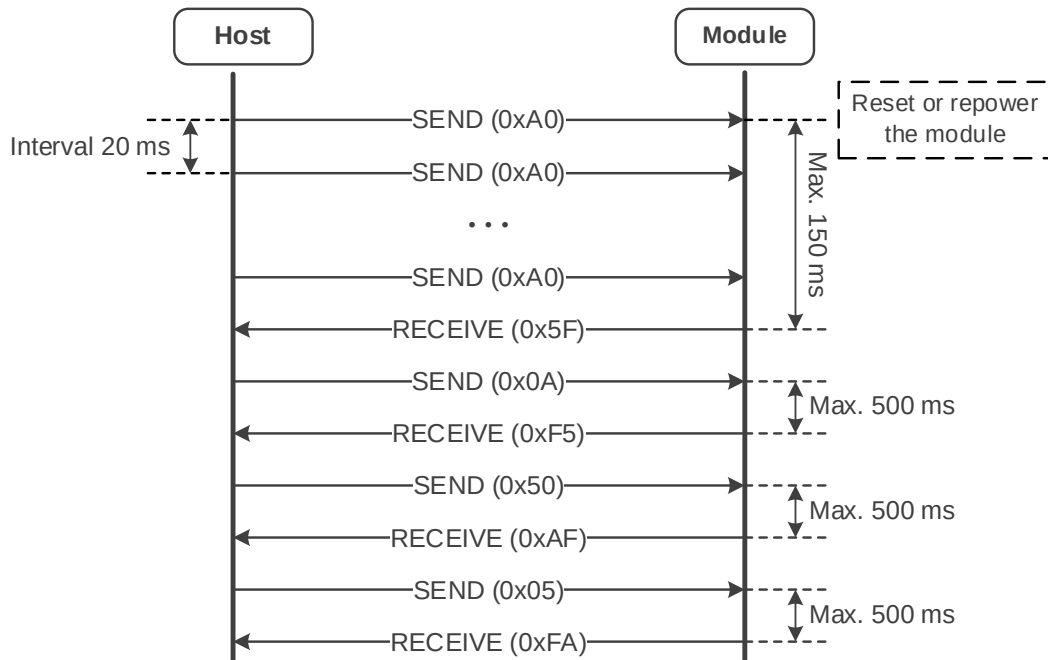


Figure 5: Handshake – UART

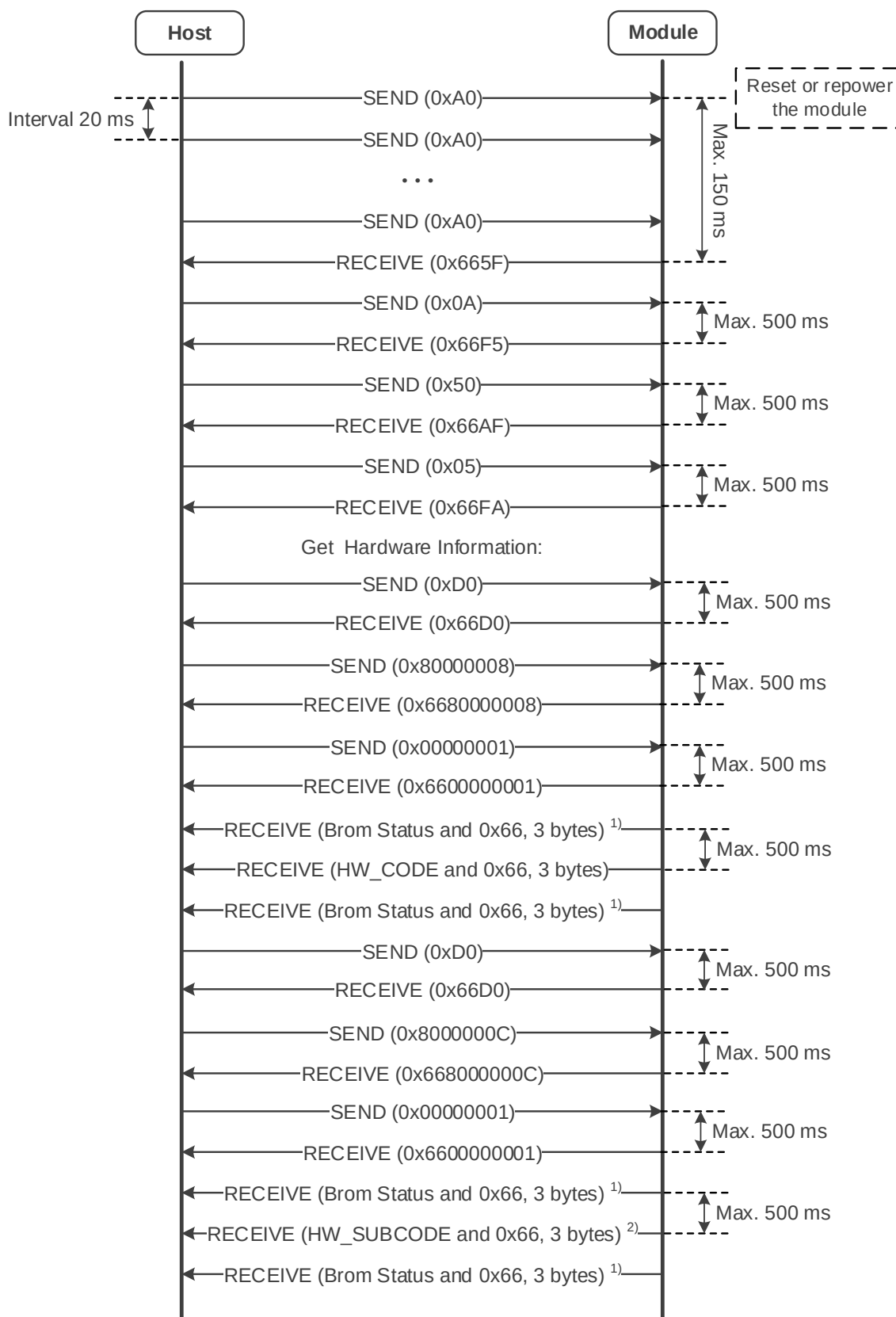


Figure 6: Handshake – I2C

NOTE

1. The host sends **0xA0** repeatedly until it receives **0x5F** via UART (**0x665F** via I2C) from the module, after which it sends **0x0A** to the module.
2. The baud rate should be 115200 bps during handshaking via UART interface.
3. ¹⁾ The value of Brom Status must be less than BROM_ERROR, otherwise this process fails (see [Table 3: List of Constants in Download Mode](#) for details).
4. ²⁾ HW_SUBCODE: A 2-byte sub-hardware information.

2.2.2. Disable Module WDT

The following diagram illustrates how to disable module WDT to avoid module reboot. For the constant parameters in the figure below, see [Table 3: List of Constants in Download Mode](#).

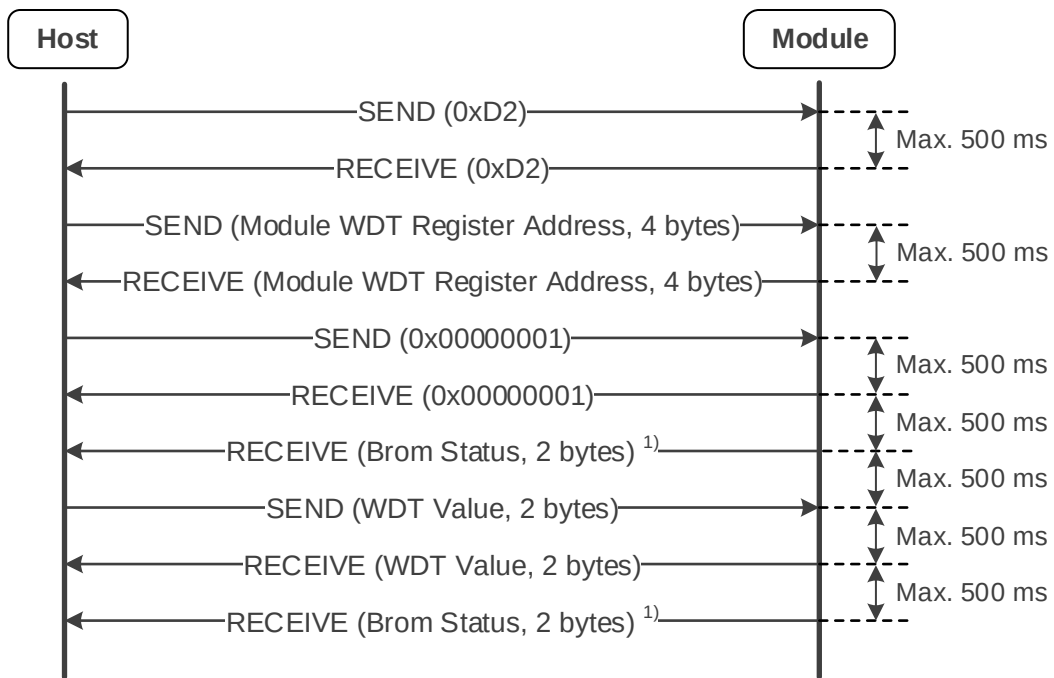


Figure 7: Disable Module WDT – UART

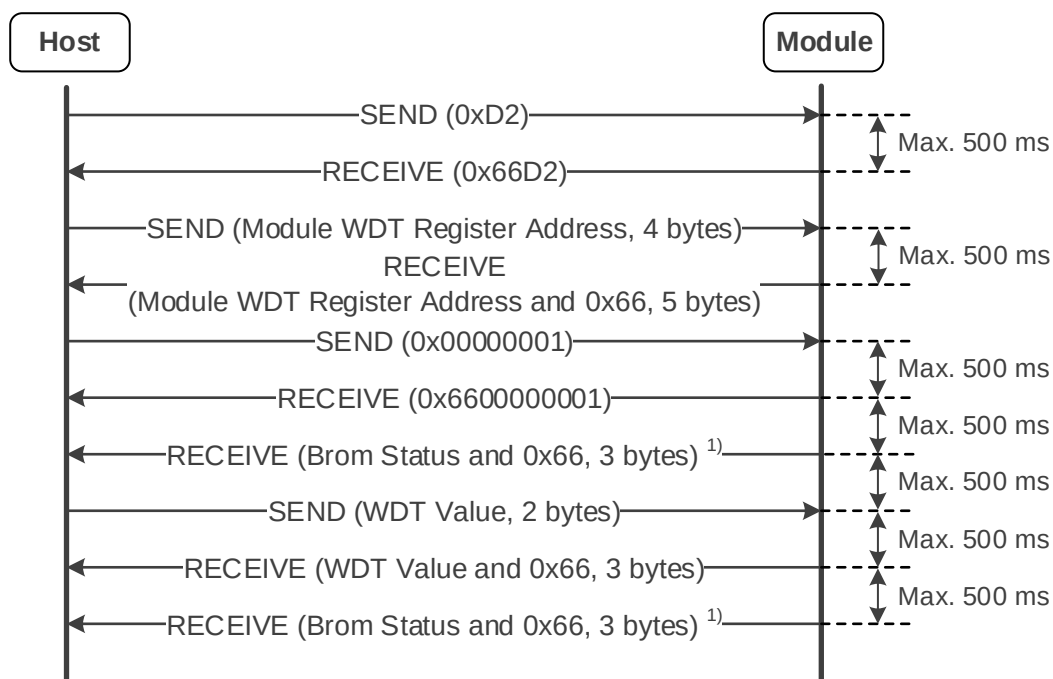


Figure 8: Disable Module WDT – I2C

NOTE

¹⁾ The value of Brom Status must be less than BROM_ERROR, otherwise this process will fail (see [Table 3: List of Constants in Download Mode](#) for details).

2.2.3. Send DA File to Module

The following diagram illustrates how to send a DA file to the module. You can use either of the two DA files provided via URAT interface: one whose transmission requires a 921600 bps baud rate, and the other whose transmission requires a 115200 bps baud rate. Or you can use the DA file provided via I2C, whose transmission requires up to 400 kbps baud rate. For the constant parameters in the figure below, see [Table 3: List of Constants in Download Mode](#).

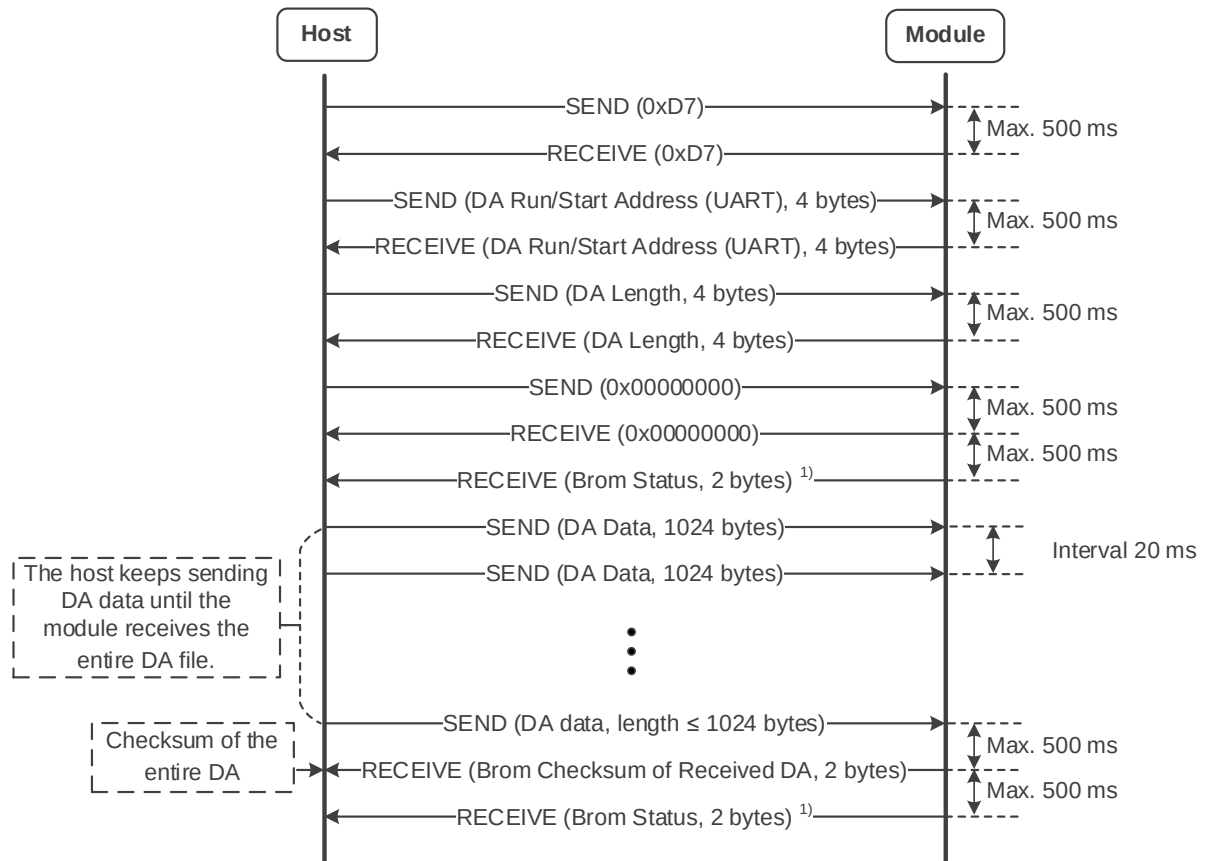


Figure 9: Send DA File to Module – UART

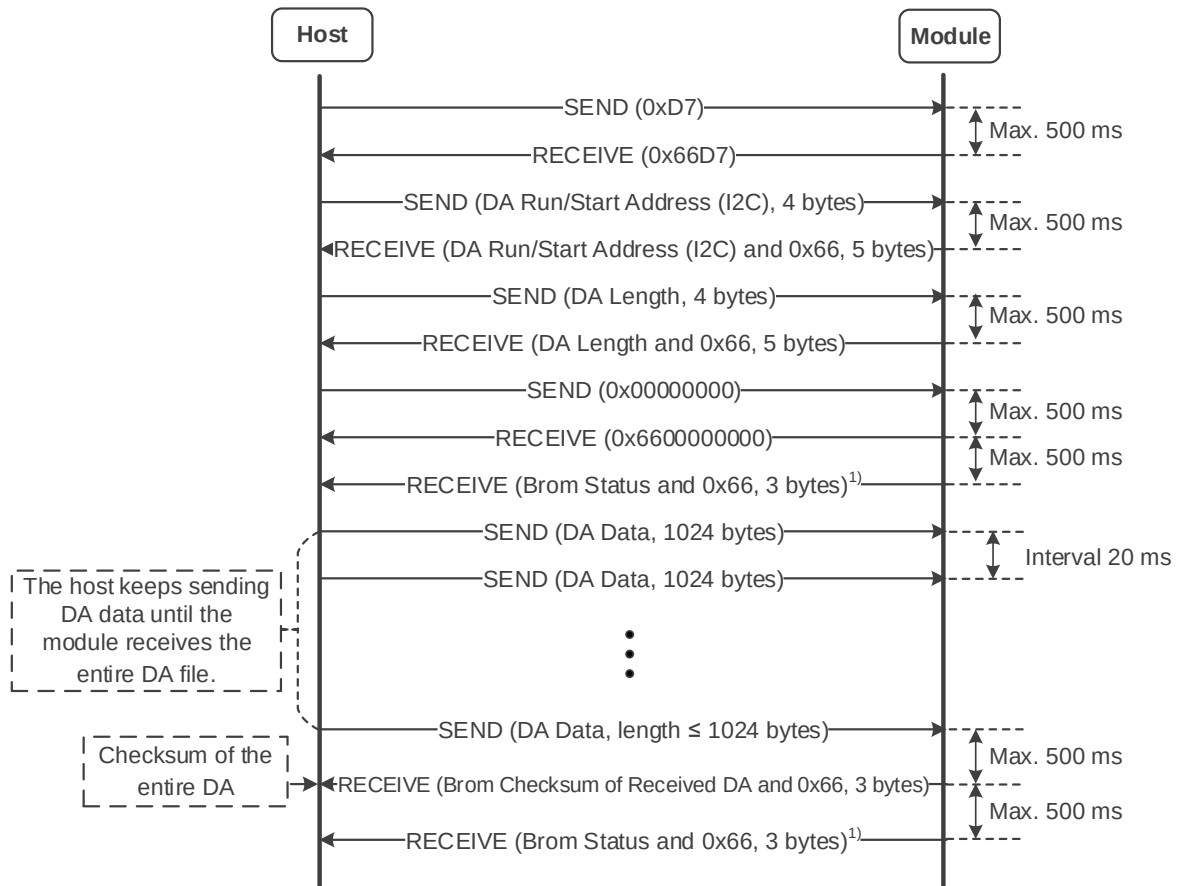


Figure 10: Send DA File to Module – I2C

NOTE

¹⁾ The value of Brom Status must be less than BROM_ERROR, otherwise this process will fail (see [Table 3: List of Constants in Download Mode](#) for details).

Checksum Code of DA File

```

uint16_t DA_compute_checksum (uint8_t *buf, uint32_t buf_len)
{
    uint16_t checksum = 0;
    if (buf == NULL || buf_len == 0) {
        return 0;
    }
    int i = 0;
    for (i = 0; i < buf_len / 2; i++) {
        checksum ^= *(uint16_t *) (buf + i * 2);
    }
    if ((buf_len % 2) == 1) {

```

```

        checksum ^= buf[i * 2];
    }
    return checksum;
}
uint16_t local_check_sum ^= DA_compute_checksum(data_buf, len);
    
```

2.2.4. Jump to DA

The following diagram illustrates how to inject the DA and how to execute it. For the constant parameters in the figure below, see [Table 3: List of Constants in Download Mode](#).

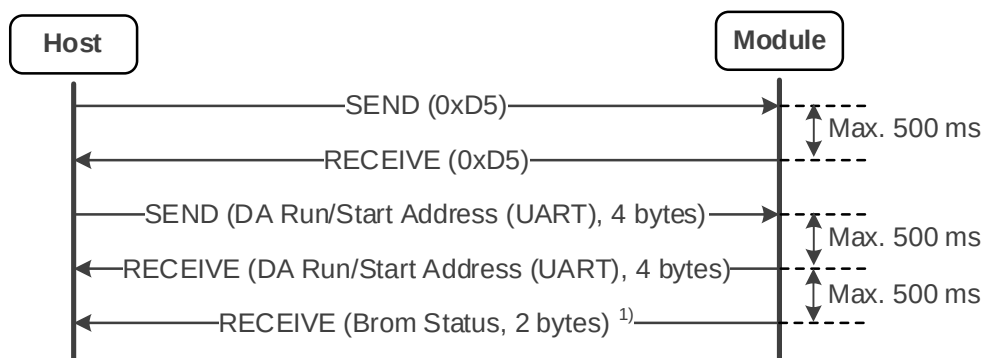


Figure 11: Jump to DA – UART

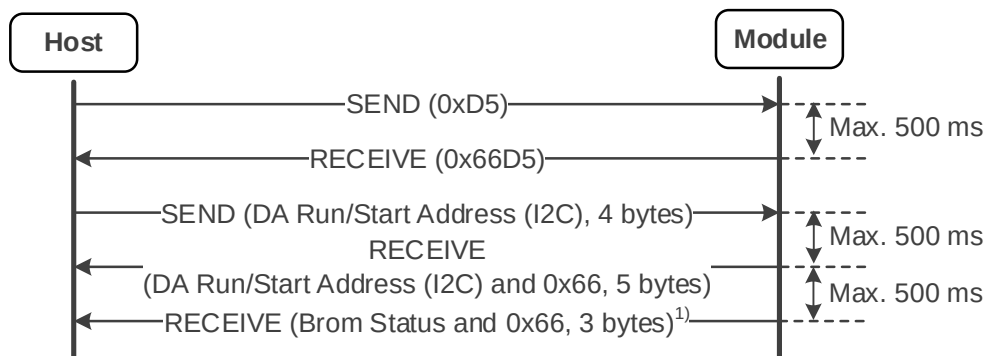


Figure 12: Jump to DA – I2C

NOTE

¹⁾ The value of Brom Status must be less than BROM_ERROR, otherwise this process will fail (see [Table 3: List of Constants in Download Mode](#) for details).

2.2.5. Synchronize with DA

The following diagram illustrates how to synchronize with DA and get the DA report. The steps of synchronizing the DA file are different between URAT and I2C. There is no need to switch the communication speed of I2C interface. For the constant parameters in the figure below, see [Table 3: List of Constants in Download Mode](#).

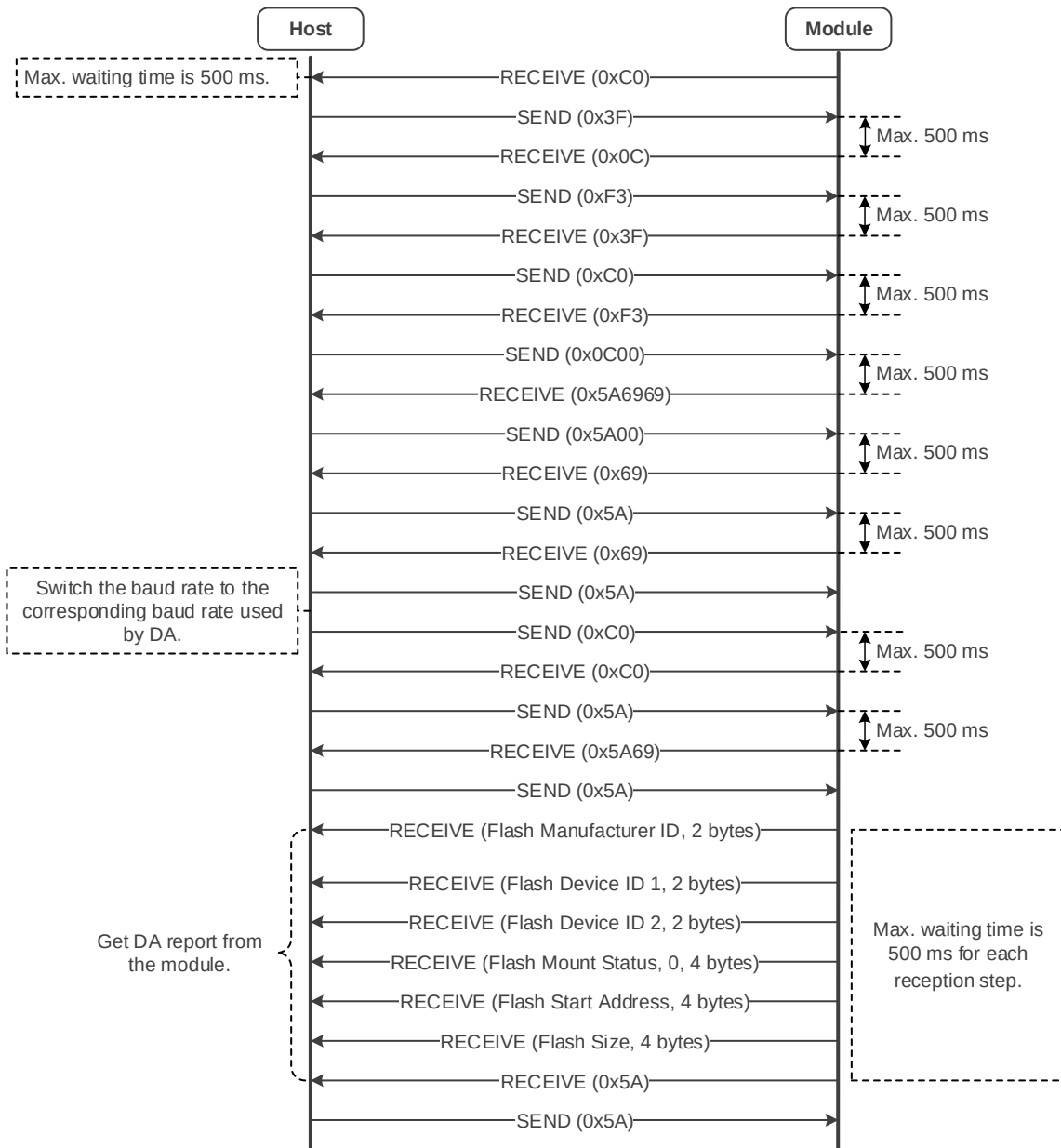


Figure 13: Synchronize with DA and Get DA Report – UART

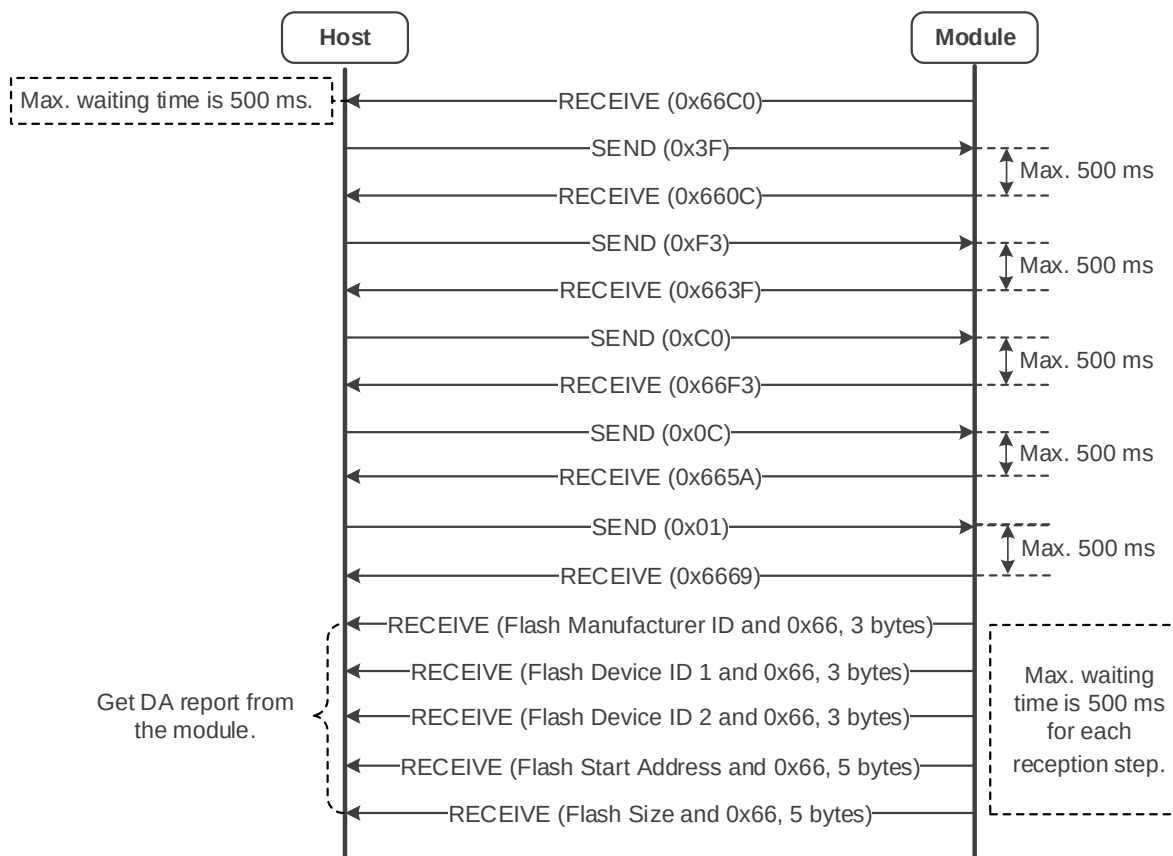


Figure 14: Synchronize with DA and Get DA Report – I2C

Table 8: Flash Parameters

Field	Length (Byte)	Description
Flash Manufacturer ID	2	Module's flash device manufacturer ID.
Flash Device ID 1	2	Module's flash device model ID 1.
Flash Device ID 2	2	Module's flash device model ID 2.
Flash Mount Status	4	Module's flash mounting status. See Table 3: List of Constants in Download Mode for details.
Flash Start Address	4	Module's flash start address. See Table 3: List of Constants in Download Mode for details.
Flash Size	4	Module's flash size. See Table 3: List of Constants in Download Mode for details.

2.2.6. Format Flash

The following diagram illustrates how to format the flash of the module. For the constant parameters in the figure below, see [Table 3: List of Constants in Download Mode](#).

2.2.6.1. Format via UART

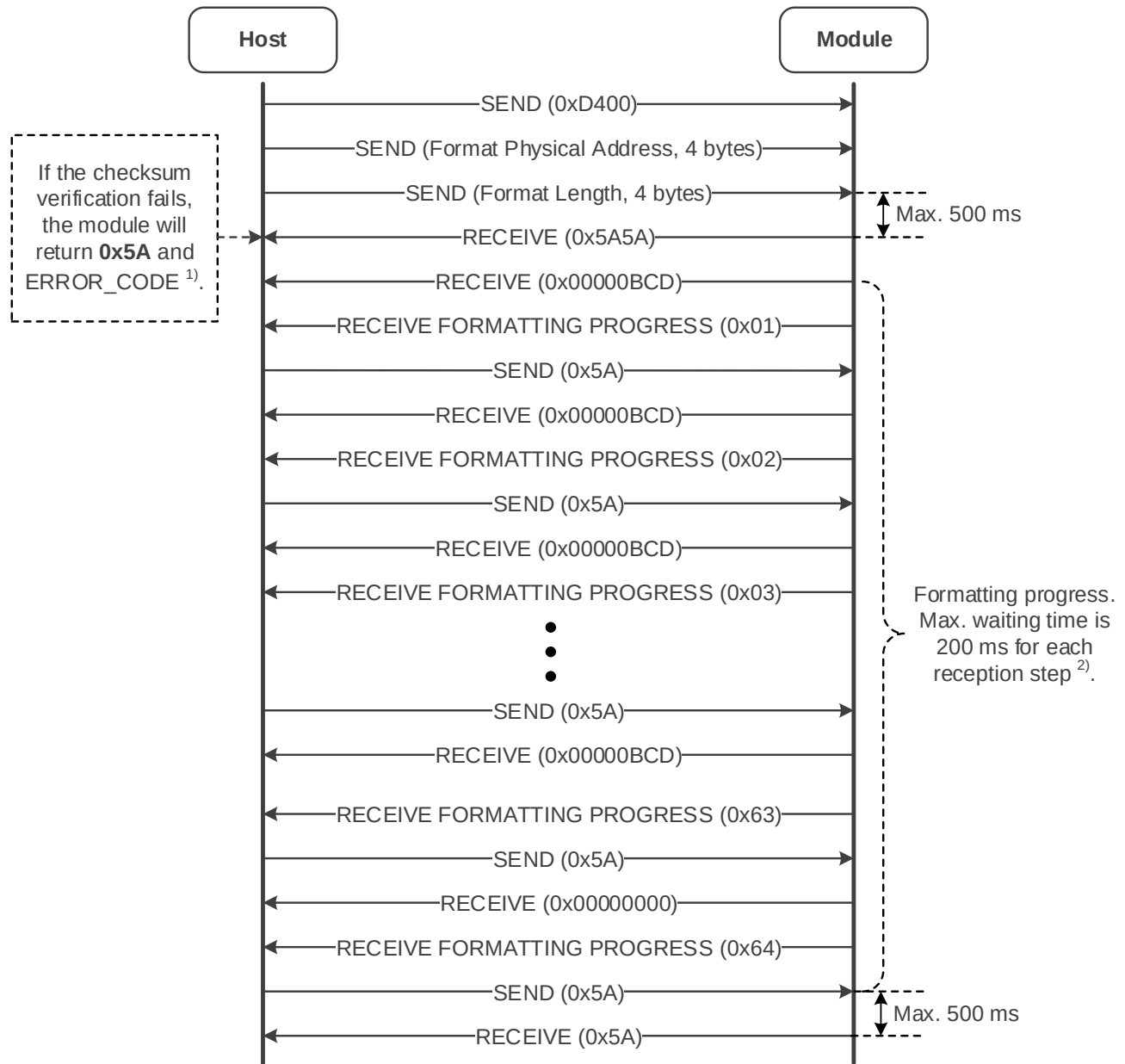


Figure 15: Format Flash – UART

NOTE

1. ¹⁾ ERROR_CODE length is four bytes.
2. ²⁾ Resend 0x5A if the host does not receive 0x00000BCD after 200 ms.

2.2.6.2. Format via I2C

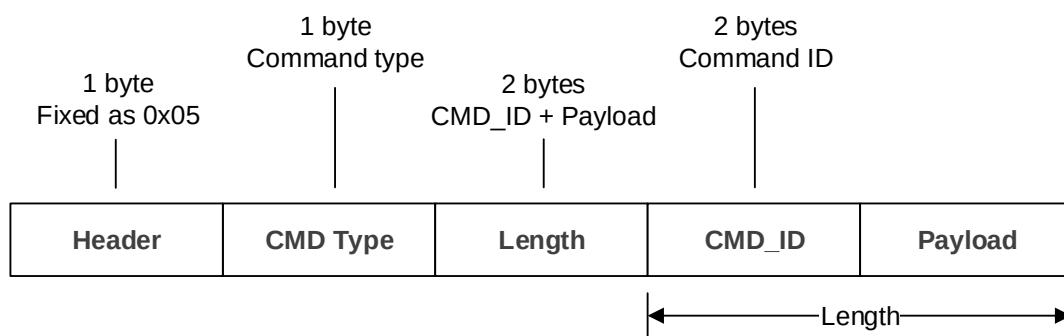


Figure 16: Binary Protocol Structure

Table 9: Description of Binary Protocol Fields

Field	Length (Byte)	Description
Header	1	Fixed as 0x05.
CMD Type	1	Command type. Sent type: 0x5A. Response type: 0x5B.
Length	2	Length of CMD_ID and Payload (without 0x66).
CMD_ID	2	Command ID.
Payload	Variable bytes	Payload data to be transferred.

Table 10: RACE_DA_FORMAT_FLASH

Header	CMD Type	Length	CMD_ID	Payload
0x05	0x5A	0x000A	0x2104	See Table 11: RACE_DA_FORMAT_FLASH Payload .

Table 11: RACE_DA_FORMAT_FLASH Payload

Byte Offset	Length (Byte)	Name	Unit	Description
0	4	Format Start Address	-	Format Start Address in each loop. The value of Format Start Address is incremented by Format Size each time through the loop.
4	4	Format Size	Byte	Format Size in each loop. The value fixed as 0x00010000.

Table 12: RACE_DA_FORMAT_FLASH_RESPONSE

Header	CMD Type	Length	CMD_ID	Payload
0x05	0x5B	0x0007	0x2104	See Table 13: RACE_DA_FORMAT_FLASH_RESPONSE Payload .

Table 13: RACE_DA_FORMAT_FLASH_RESPONSE Payload

Byte Offset	Length (Byte)	Name	Unit	Description
0	1	Status	-	Response status. 0x00 = Success Others = Failure
1	4	Format Start Address	-	Format Start Address in each loop. The value of Format Start Address is incremented by Format Size each time through the loop.

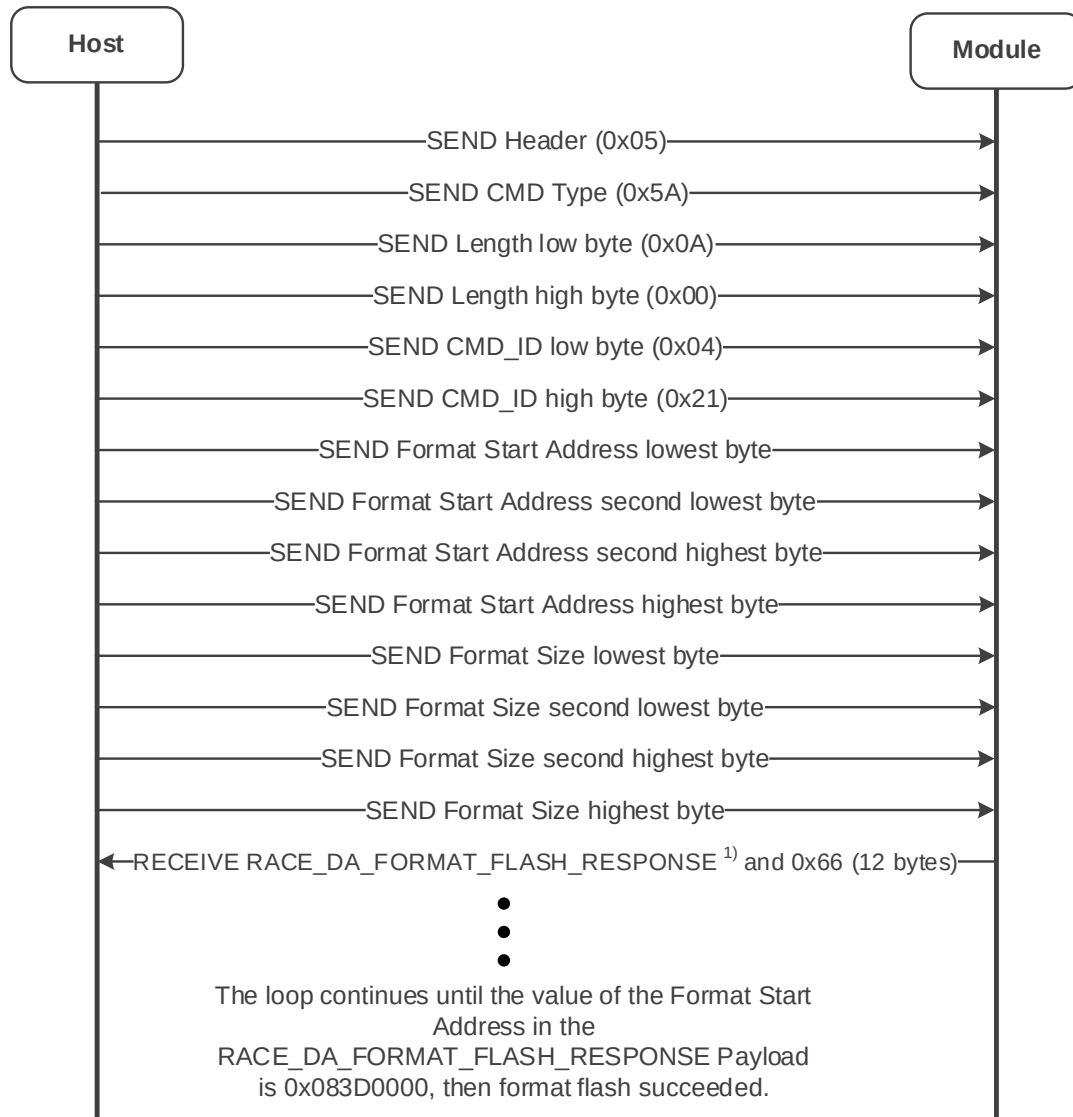


Figure 17: Format Flash – I2C

NOTE

¹⁾ RACE_DA_FORMAT_FLASH_RESPONSE see [Table 12: RACE DA FORMAT FLASH RESPONSE](#) and [Table 13: RACE DA FORMAT FLASH RESPONSE Payload](#).

2.2.7. Send FW File to Module

The following diagram illustrates how to send FW files to the module. For the constant parameters in the figure below, see [Table 3: List of Constants in Download Mode](#).

2.2.7.1. Send FW File via UART

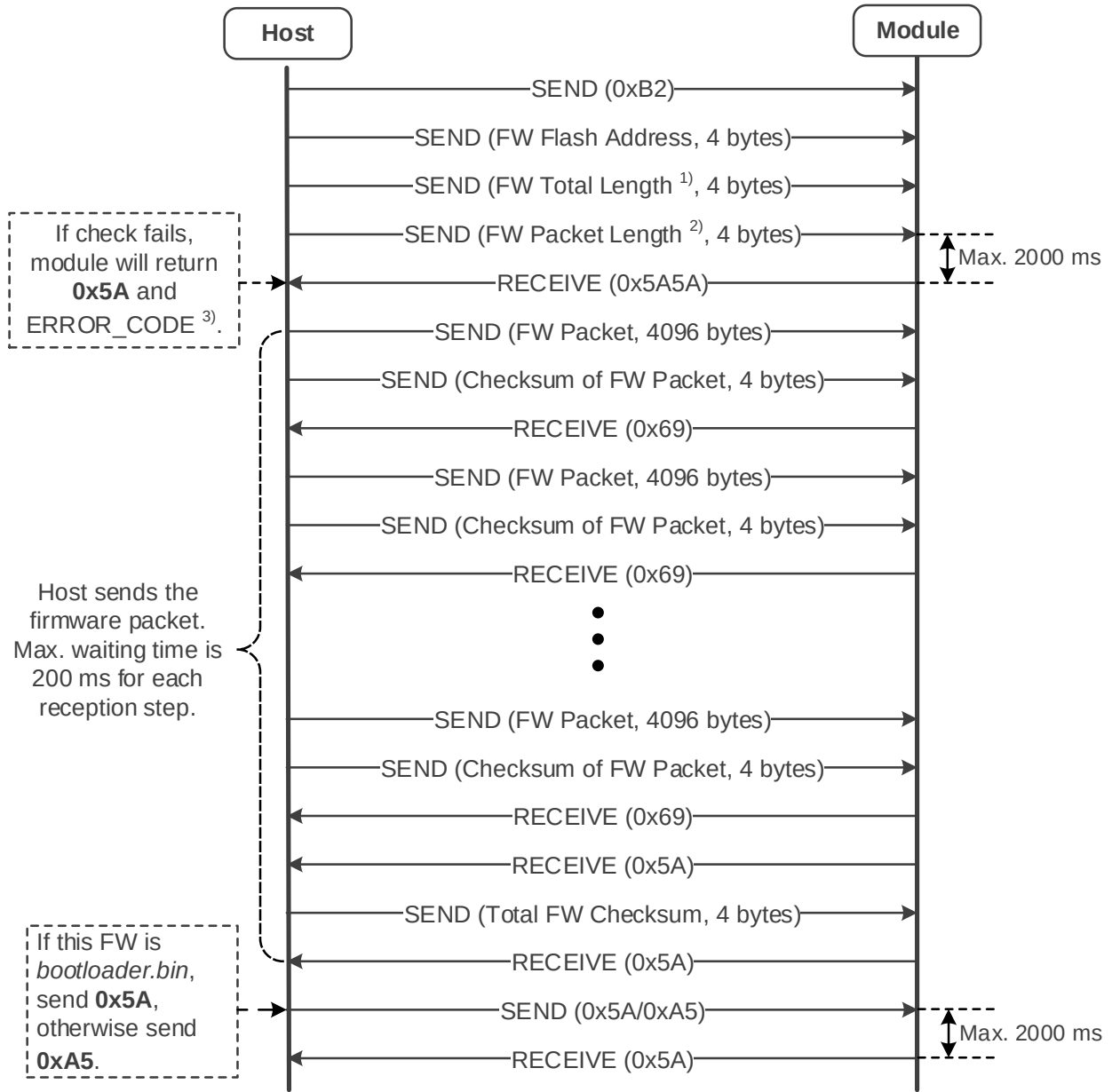


Figure 18: Send FW File to Module – UART

NOTE

1. ¹ FW total length must be an integer multiple of 4K.
2. ² Size of each packet is 4096 bytes.
3. ³ **ERROR_CODE** length is four bytes.

2.2.7.2. Send FW File via I2C

For the binary protocol structure for sending FW via I2C, see [Chapter 2.2.6.2 Format via I2C](#).

Table 14: RACE_DA_WRITE_DATA

Header	CMD Type	Length	CMD_ID	Payload
0x05	0x5A	0x100C	0x2100	See Table 15: RACE_DA_WRITE_DATA Payload .

Table 15: RACE_DA_WRITE_DATA Payload

Byte Offset	Length (Byte)	Name	Unit	Description
0	4	Data Start Address	-	Data Start Address in each loop. The value of Data Start Address incremented by Data Length each time through the loop.
4	2	Data Length	Byte	The value fixed as 0x1000.
6	4096	Data	-	Data of FW packet.
4102	4	Checksum of Data	-	Checksum of Data. See Checksum Code of FW Packet for details.

Table 16: RACE_DA_WRITE_DATA_RESPONSE

Header	CMD Type	Length	CMD_ID	Payload
0x05	0x5B	0x0007	0x2100	See Table 17: RACE_DA_WRITE_DATA_RESPONSE Payload .

Table 17: RACE_DA_WRITE_DATA_RESPONSE Payload

Byte Offset	Length (Byte)	Name	Unit	Description
0	1	Status	-	Response status. 0x00 = Success Others = Failure
1	4	Data Start Address	-	Data Start Address in each loop. The value of Data Start Address incremented by Data Length each time through the loop.

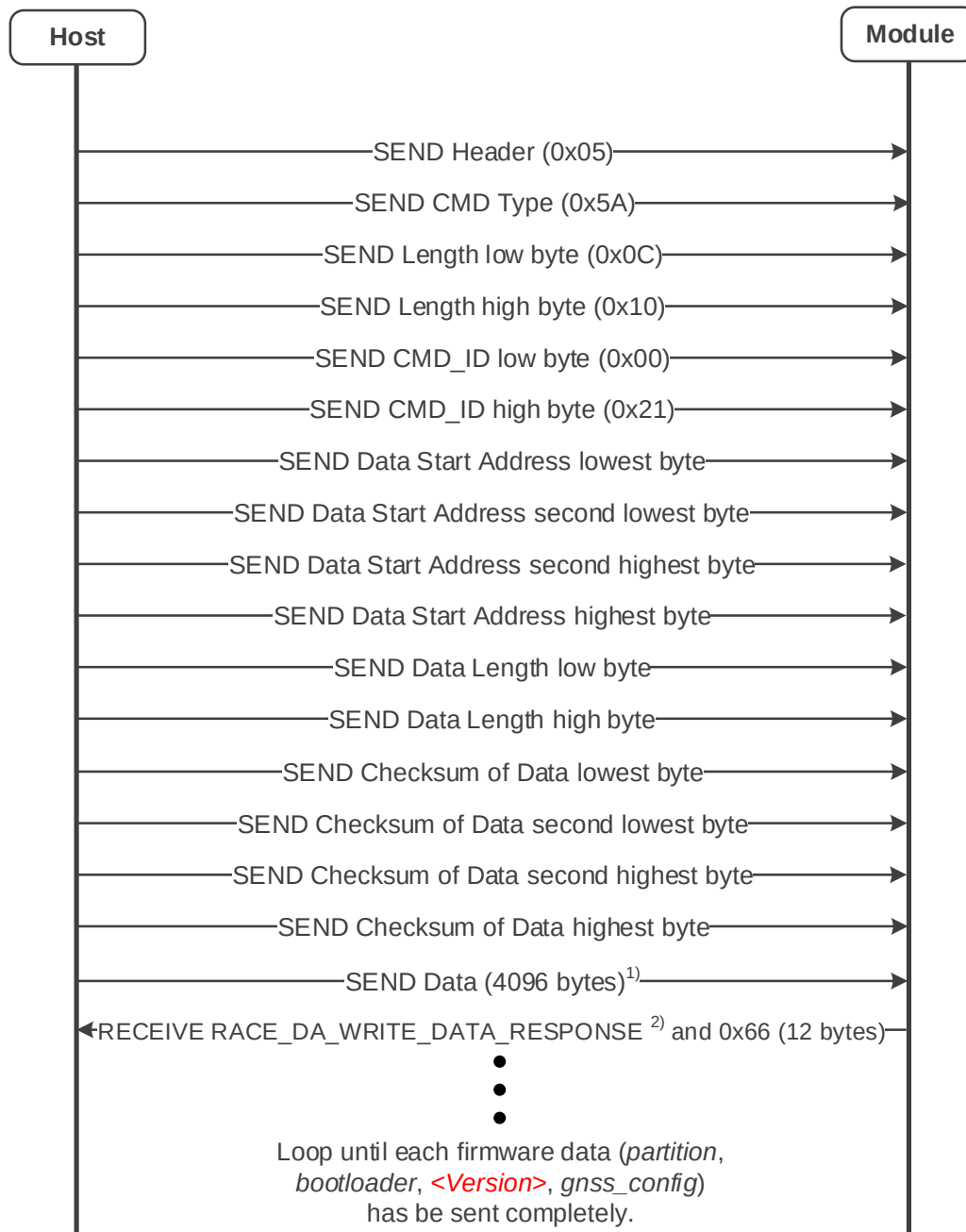


Figure 19: Send FW File to Module – I2C

NOTE

1. ¹⁾ After jumping to DA, the module's I2C interface can only receive up to 256 bytes at a time, so data must be transmitted in packets.
2. ²⁾ For receive data frames, see [Figure 16: Binary Protocol Structure](#), [Table 16: RACE DA WRITE DATA RESPONSE](#) and [Table 17: RACE DA WRITE DATA RESPONSE Payload](#).

Checksum Code for FW Packet

```
uint32_t CRC_compute_simple_checksum (uint8_t *buf, uint32_t buf_len) {
    uint32_t checksum = 0;
    if (buf == NULL || buf_len == 0) {
        return 0;
    }
    for (int i = 0; i < buf_len; i++) {
        checksum += *(buf + i);
    }
    return checksum;
}

uint32_t packet_checksum = CRC_compute_simple_checksum (buf, buf_len);
//Total Checksum Code of the FW file:(I2C no need)
uint32_t Total_FW_checksum += packet_checksum;
```

NOTE

If the last FW packet is less than 4096 bytes, fill in with "0xFF" at the end of the last FW packet. The value of the calculated checksum includes the additional "0xFF".

2.3. Upgrade Mode

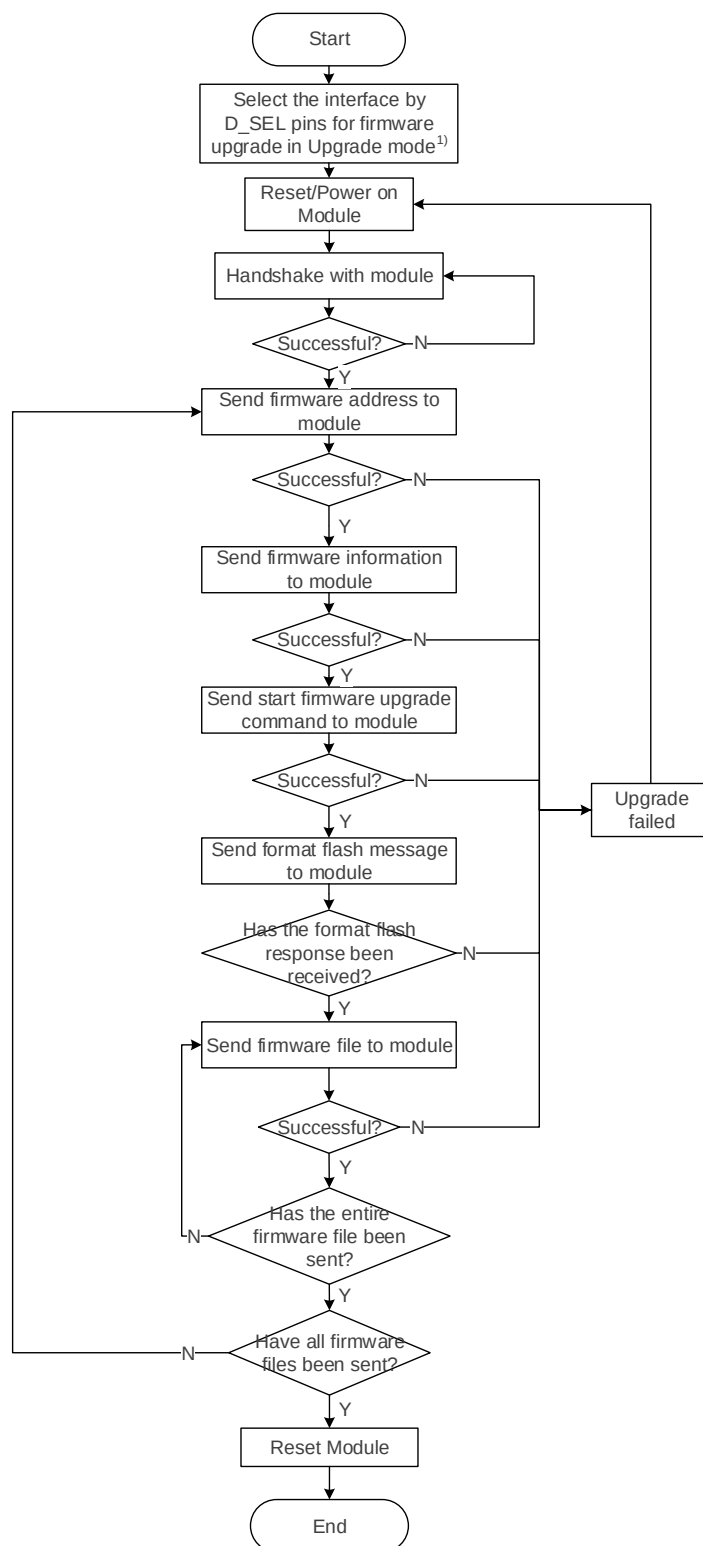


Figure 20: Communication Process in Upgrade Mode

NOTE

¹⁾ Currently, only LC29H (AI) supports firmware upgrade in Upgrade mode via either I2C or SPI by selecting the interface:

- D_SEL1 = 1, D_SEL2 = 0: select SPI
- D_SEL1 = 0, D_SEL2 = 0: select I2C interface (default)

LC79H (AL) only supports firmware upgrade in Upgrade mode through the I2C interface and selecting the interface is not involved in the above process. Other modules do not support firmware upgrade in Upgrade mode.

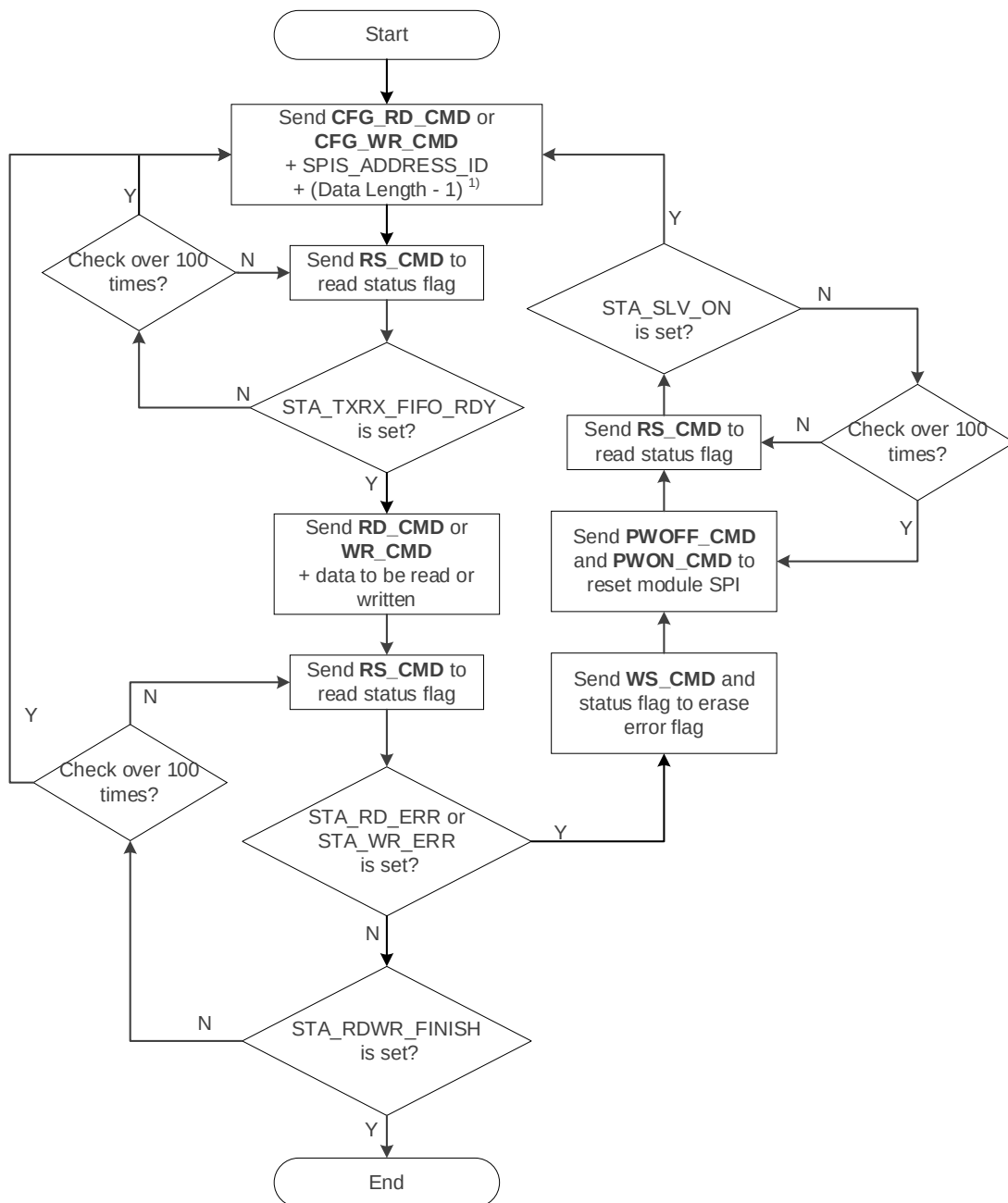


Figure 21: Module SPI Read/Write Flow

NOTE

¹⁾ The Data Length is the size (in byte) of the data to be read or written, the actual value to be transmitted should be (Data Length - 1). For more details, see [document \[4\] SPI application note](#).

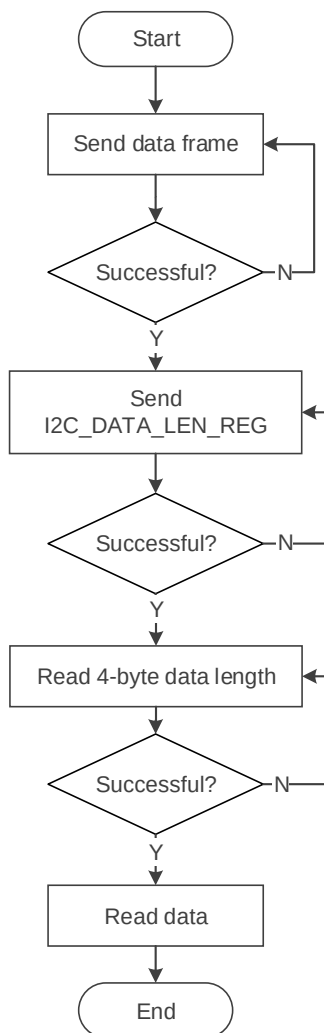


Figure 22: Module I2C Read/Write Flow

The following sections illustrate how the host establishes communication with the module and sends commands and files to the module in Upgrade mode:

- Handshake with the module
- Send firmware address to the module
- Send firmware information to the module
- Send start firmware upgrade command to the module
- Send format flash message to module
- Send FW (firmware) file to the module

NOTE

The module will output firmware version information automatically after each boot-up. For example, LC79H (AL) will output **\$PQTMVER,1,MODULE,LC79HALNR11A03S,2024/03/07,08:59:25*14** after its boot-up. You can also execute **\$PQTMVERNO*58** to query the firmware version information. For more details, see [documents \[1\]](#), [\[2\]](#), and [\[3\] protocol specifications](#).

2.3.1. Handshake with Module

The handshaking mechanism for firmware upgrade in Upgrade mode is as follows:

If the host sends handshake data 1 (HANDSHAKE_WORD1) and the module responds with handshake response data 1 (RSP_WORD1) within 100 ms, followed by the host sending handshake data 2 (HANDSHAKE_WORD2) and receiving handshake response data 2 (RSP_WORD2) within 2000 ms, the handshake succeeds.

Not receiving RSP_WORD1 within 100 ms or RSP_WORD2 within 2000 ms indicates a handshake failure, requiring a module reset and re-initiating the handshake.

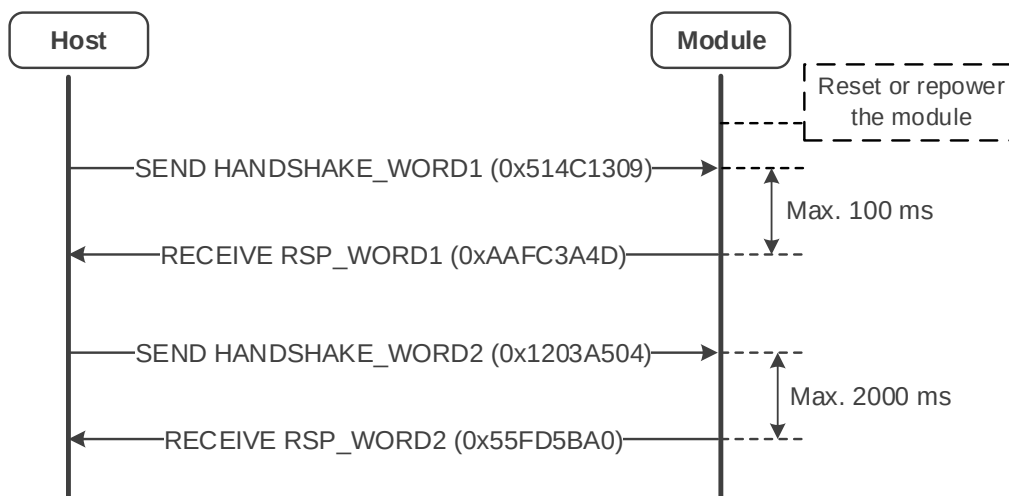
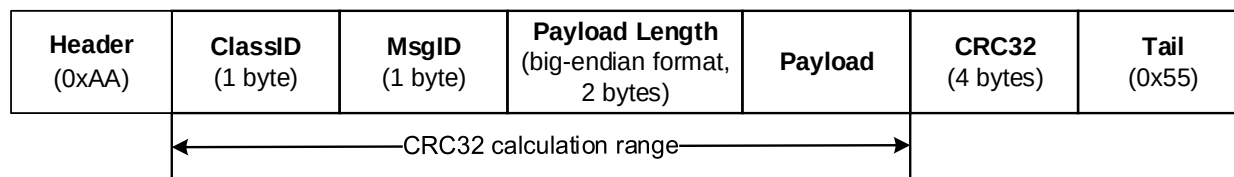


Figure 23: Handshake

2.3.2. Send Firmware Address to Module

The files transmitted in Upgrade mode are *partition_table.bin*, *<Version>.bin*, and *gnss_config.bin*. Refer to [Table 6: Format Length and FW Flash Address \(General Applications\)](#) for the firmware flash address.

In Upgrade mode, the protocol structure for data transmission is as follows.


Figure 24: Protocol Structure in Upgrade Mode
Table 18: Protocol Structure in Upgrade Mode

Field	Description
Header	The frame header is 0xAA.
ClassID	Class ID. See Table 19: ClassID and MsgID for details.
MsgID	Message ID. See Table 19: ClassID and MsgID for details.
Payload Length	The length of Payload, 2 bytes (in big-endian format).
Payload	The content of the message.
CRC32	4 bytes (in big endian format).
Tail	The frame tail is 0x55.

Table 19: ClassID and MsgID

ClassID	MsgID	Description
0x02	0x01	Host sends firmware address.
	0x02	Host sends firmware information.
	0x03	Host sends command to start firmware upgrade.
	0x04	Host sends firmware data.
	0x21	Host requests to format flash.
	0x00	Module's response.

Table 20: Module Status in Upgrade Mode

Status (2 Bytes)	Description
0x0000	Executed successfully.
0x0001	Unknow error.
0x0002	CRC32 error.
0x0003	Timeout error.
0x0004	Unsupported message.
0x0005	Frame format error.
0x0020	Flash erase error.
0x0021	Flash write error.
0x0041	Requested format error.

NOTE

Module status is transmitted in big-endian format.

The steps for the host sending firmware address to the module in Upgrade mode are as follows:

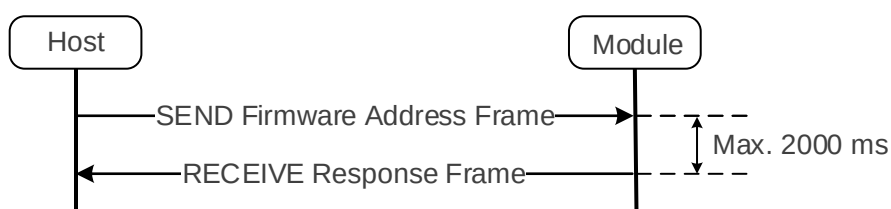


Figure 25: Send Firmware Address

Table 21: Firmware Address Frame from Host

Header	ClassID	MsgID	Payload Length	Payload	CRC32	Tail
0xAA	0x02	0x01	0x0004	See Table 6: Format Length and FW Flash Address (General Applications) .	4 bytes	0x55

Table 22: Firmware Address Response Frame from Module

Header	ClassID	MsgID	Payload Length	Payload			CRC32	Tail
				ClassID	MsgID	Status		
0xAA	0x02	0x00	0x0004	0x02	0x01	See Table 20: Module Status in Upgrade Mode.	4 bytes	0x55

NOTE

Firmware address is transmitted in big-endian format.

2.3.3. Send Firmware Information to Module

For the protocol structure for sending firmware information, see [Chapter 2.3.2 Send Firmware Address to Module](#).

The steps for the host sending firmware information to the module in Upgrade mode are as follows:

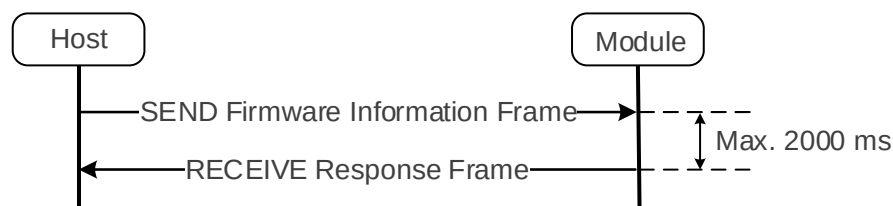


Figure 26: Send Firmware Information

Table 23: Firmware Information Frame from Host

Header	ClassID	MsgID	Payload Length	Payload	CRC32	Tail
0xAA	0x02	0x02	0x0010	See Table 24: Firmware Information Parameters.	4 bytes	0x55

Table 24: Firmware Information Parameters

Parameter	Length (Byte)	Description
<FWSize>	4	Firmware size, that is, the size of the .bin file to be sent.

Parameter	Length (Byte)	Description
<FWCRC32>	4	CRC32 of the firmware size (calculated in little-endian mode) and the entire firmware.
<DestAddr>	4	Firmware start address. See Table 6: Format Length and FW Flash Address (General Applications) .
<EraseFlag>	1	Erase flag must be 0x01.
<Reserved>	3	Reserved. Keep it as 0x00.

NOTE

<FWSize>, <FWCRC32>, and <DestAddr> are transmitted in big-endian format.

Table 25: Firmware Information Response Frame from Module

Header	ClassID	MsgID	Payload Length	Payload			CRC32	Tail
				ClassID	MsgID	Status		
0xAA	0x02	0x00	0x0004	0x02	0x02	See Table 20: Module Status in Upgrade Mode .	4 bytes	0x55

2.3.4. Send Start Firmware Upgrade Command to Module

For the protocol structure for sending command to start firmware upgrade, see [Chapter 2.3.2 Send Firmware Address to Module](#).

The steps for the host sending start firmware upgrade data to the module in Upgrade mode are as follows:

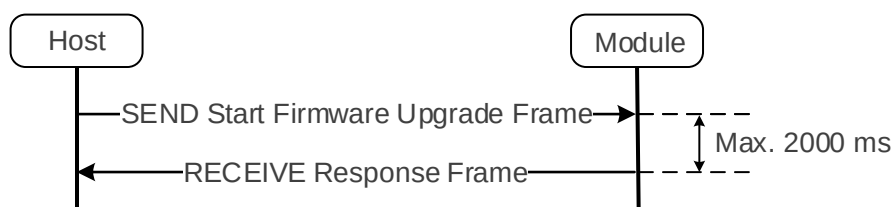


Figure 27: Send Start Firmware Upgrade Command

Table 26: Start Firmware Upgrade Frame from Host

Header	ClassID	MsgID	Payload Length	Payload	CRC32	Tail
0xAA	0x02	0x03	0x0000	-	4 bytes	0x55

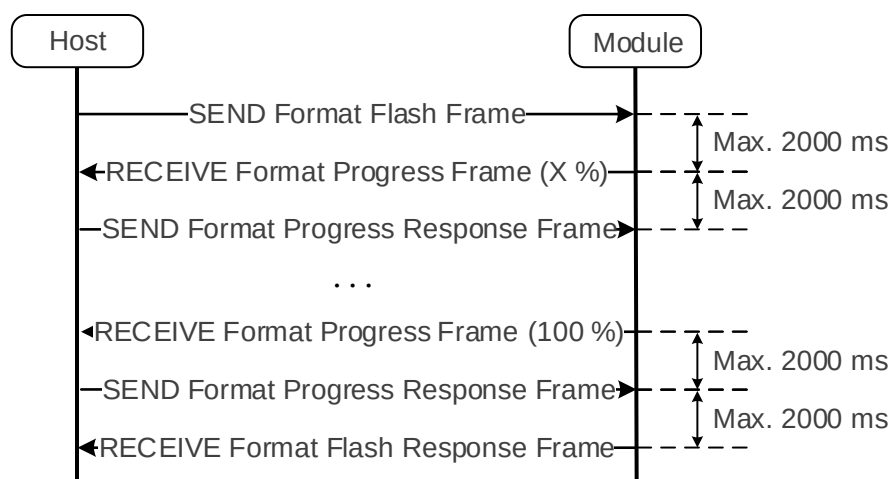
Table 27: Start Firmware Upgrade Response Frame from Module

Header	ClassID	MsgID	Payload Length	Payload			CRC32	Tail
				ClassID	MsgID	Status		
0xAA	0x02	0x00	0x0004	0x02	0x03	See Table 20: Module Status in Upgrade Mode .	4 bytes	0x55

2.3.5. Send Format Flash Message to Module

For the protocol structure for formatting flash, see [Chapter 2.3.2 Send Firmware Address to Module](#).

The steps for the host formatting flash in Upgrade mode are as follows:


Figure 28: Format Flash

NOTE

Since it takes some time for the module to erase data, this process requires the host to read the erasure progress in a loop until the process is fully completed.

Table 28: Format Flash Frame from Host

Header	ClassID	MsgID	Payload Length	Payload	CRC32	Tail
0xAA	0x02	0x21	0x0008	See Table 29: Format Flash Frame Parameters .	4 bytes	0x55

Table 29: Format Flash Frame Parameters

Parameter	Length (Byte)	Description
<StartAddr>	4	Firmware start address. See Table 6: Format Length and FW Flash Address (General Applications) .
<FormatLength> ¹⁾	4	Format length, equals the value of <FWSize>. See Table 24: Firmware Information Parameters .

NOTE

- ¹⁾ When sending <Version>.bin, the <FormatLength> should be the difference between the flash address of gnss_config.bin and the flash address of <Version>.bin.
- <StartAddr> and <FormatLength> are transmitted in big-endian format.

Table 30: Format Progress Frame from Module

Header	ClassID	MsgID	Payload Length	Payload				CRC32	Tail
				ClassID	MsgID	Progress	Status		
0xAA	0x02	0x00	0x0005	0x02	0x21	1 byte	See Table 20: Module Status in Upgrade Mode .	4 bytes	0x55

Table 31: Format Progress Response Frame from Host

Header	ClassID	MsgID	Payload Length	Payload			CRC32	Tail
				Reserved	Progress	Status		
0xAA	0x02	0x21	0x0005	0x0000	1 byte	See Table 20: Module Status in Upgrade Mode .	4 bytes	0x55

Table 32: Format Flash Response Frame from Module

Header	ClassID	MsgID	Payload Length	Payload			CRC32	Tail
				ClassID	MsgID	Status		
0xAA	0x02	0x00	0x0004	0x02	0x21	See Table 20: Module Status in Upgrade Mode .	4 bytes	0x55

2.3.6. Send FW File to Module

For the protocol structure for sending FW files, see [Chapter 2.3.2 Send Firmware Address to Module](#).

The steps for the host sending FW file to the module in Upgrade mode are as follows:

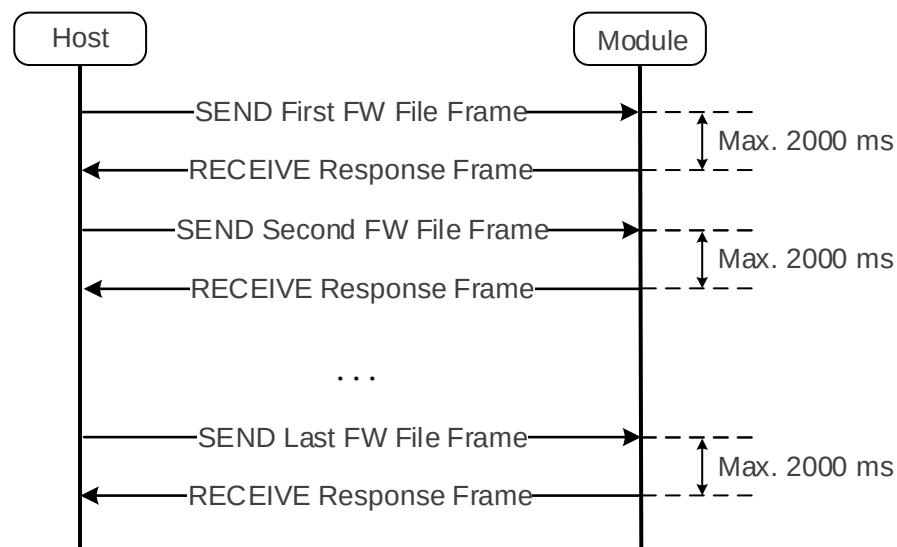


Figure 29: Send FW Flow

Table 33: FW File Frame from Host

Header	ClassID	MsgID	Payload Length	Payload		CRC32	Tail
				Packet Number	FW File Data		
0xAA	0x02	0x04	0x1004	4 bytes	Max. 4096 bytes ¹⁾	4 byte	0x55

Table 34: FW File Response Frame from Module

Header	ClassID	MsgID	Payload Length	Payload			CRC32	Tail
				ClassID	MsgID	Status		
0xAA	0x02	0x00	0x0004	0x02	0x04	See Table 20: Module Status in Upgrade Mode .	4 bytes	0x55

NOTE

1. Packet Number is in big-endian format and starts from 0.
2. ¹⁾ If the last FW packet is less than 4096 bytes, fill in with "0xFF" at the end of the last FW packet. The value of the calculated checksum includes the additional "0xFF".
3. In Upgrade mode, only one file can be transmitted at a time. After the current file transmission is completed, you must return to the steps described in [Chapter 2.3.2 Send Firmware Address to Module](#) to proceed with the transmission of the next file. Once all files have been transmitted, you can directly reset the module to run on the new firmware.
4. Note that in Upgrade mode, when performing firmware upgrades via the I2C interface, the maximum data frame size that the module can receive is 256 bytes. Therefore, when sending firmware files, it is necessary to send the data in frames of 256 bytes. If the last frame is less than 256 bytes, simply send it as is.

3 Upgrade Implementation Examples

3.1. Download Mode – Firmware Upgrade via UART Interface

Below is an example of LC29H (BA) firmware upgrading via UART interface.

[//Figure 5: Handshake – UART](#)

//Host keeps sending 0xA0. Reset the module.

A0

A0

A0

...

A0

A0

A0

A0

//Module responds with 0x5F.

5F

//Host sends 0x0A.

0A

//Module responds with 0xF5.

F5

//Host sends 0x50.

50

//Module responds with 0xAF.

AF

//Host sends 0x05.

05

//Module responds with 0xFA.

FA

[//Figure 7: Disable Module WDT – UART](#)

//Host sends 0xD2.

D2

//Module responds with 0xD2.

D2

//Host sends 0xA2080000.

```

A2 08 00 00
//Module responds with 0xA2080000.
A2 08 00 00
//Host sends 0x00000001.
00 00 00 01
//Module responds with 0x00000001.
00 00 00 01
//Module responds with Brom Status.
00 01
//Host sends 0x0010.
00 10
//Module responds with 0x0010.
00 10
//Module responds with Brom Status.
00 01
//Figure 9: Send DA File to Module – UART
//Host sends 0xD7.
D7
//Module responds with 0xD7.
D7
//Host sends DA Run/Start Address.
04 20 40 00
//Module responds with DA Run/Start Address.
04 20 40 00
//Host sends DA Length. (For DA file whose transmission requires a 115200 bps baud rate, the DA
Length is “00 00 6C F7”; for DA file whose transmission requires a 921600 bps baud rate, the DA Length
is “00 00 6D CB”.)
00 00 6C F7
//Module responds with DA Length.
00 00 6C F7
//Host sends 0x00000000.
00 00 00 00
//Module responds with 0x00000000.
00 00 00 00
//Module responds with Brom Status.
00 00
//Host sends DA Data, in a chunk of 1024 bytes.
27 B6 11 49 11 4A 4F F0 00 00 91 42 00 F0 05 80 41 F8 04 0B 91 42 7F F6 .....
//Host sends DA Data, in a chunk of 1024 bytes.
23 48 08 3E 1F 46 89 46 92 46 FF F7 95 FF 10 23 33 60 34 68 1F 4D E3 06 .....
.....
//Host sends the last chunk of DA Data (length ≤ 1024 bytes).
65 73 73 28 30 78 25 78 29 20 74 6F 20 62 6C 6F 63 6B 20 69 6E 64 65 78 .....
//Module responds with Brom Checksum of Received DA (2 bytes). (Checksum Code of DA File)

```

46 1D

//Module responds with Brom Status.

00 00

//[Figure 11: Jump to DA – UART](#)

//Host sends **0xD5**.

D5

//Module responds with **0xD5**.

D5

//Host sends DA Run/Start Address.

04 20 40 00

//Module responds with DA Run/Start Address.

04 20 40 00

//Module responds with Brom Status.

00 00

//[Figure 13: Synchronize with DA and Get DA Report – UART](#)

//Module responds with **0xC0**.

C0

//Host sends **0x3F**.

3F

//Module responds with **0x0C**.

0C

//Host sends **0xF3**.

F3

//Module responds with **0x3F**.

3F

//Host sends **0xC0**.

C0

//Module responds with **0xF3**.

F3

//Host sends **0x0C00**.

0C 00

//Module responds with **0x5A6969**.

5A 69 69

//Host sends **0x5A00**.

5A 00

//Module responds with **0x69**.

69

//Host sends **0x5A**.

5A

//Module responds with **0x69**.

69

//Host sends **0x5A**.

5A

//Host sends **0xC0**. (For the DA file whose transmission requires a 921600 bps baud rate, change the

UART baud rate to 921600 bps; for the other DA file, keep the 115200 bps baud rate.)

C0

//Module responds with **0xC0**.

C0

//Host sends **0x5A**.

5A

//Module responds with **0x5A69**.

5A 69

//Host sends **0x5A**.

5A

//Module responds with Flash Manufacturer ID.

00 EF

//Module responds with Flash Device ID 1.

00 60

//Module responds with Flash Device ID 2.

00 16

//Module responds with Flash Mount Status.

00 00 00 00

//Module responds with Flash Start Address.

08 00 00 00

//Module responds with Flash Size.

00 40 00 00

//Module responds with **0x5A**.

5A

//Host sends **0x5A**.

5A

//[Figure 15: Format Flash – UART](#)

//Host sends **0xD400**.

D4 00

//Host sends Format Physical Address.

08 00 00 00

//Host sends Format Length. See [Table 6: Format Length and FW Flash Address \(General Applications\)](#).

00 3E 00 00

//Module responds with **0x5A5A**.

5A 5A

//Module responds with **0x00000BCD** and formatting progress.

00 00 0B CD 01

//Host sends **0x5A** and waits for the module to respond with **0x00000BCD** and formatting progress.

5A 00 00 0B CD 02

//Host sends **0x5A** and waits for the module to respond with **0x00000BCD** and formatting progress.

5A 00 00 0B CD 03

//Host sends **0x5A** and waits for the module to respond with **0x00000BCD** and formatting progress.

5A 00 00 0B CD 04

.....

//Host sends **0x5A** and waits for the module to respond with **0x0000BCD** and formatting progress.

5A 00 00 0B CD 63

//Host sends **0x5A** and waits for the module to respond with **0x00000000** and formatting progress.

5A 00 00 00 00 64

//Host sends **0x5A**.

5A

//Module responds with **0x5A**.

5A

//[Figure 18: Send FW File to Module – UART](#)

//Host sends **0xB2**.

B2

//Host sends FW Flash Address (*partition_table.bin*).

08 00 00 00

//Host sends FW Total Length (*partition_table.bin*). (4K alignment: FW total length must be an integer multiple of 4K.)

00 00 10 00

//Host sends FW Packet Length (4 bytes).

00 00 10 00

//Module responds with **0x5A5A**.

5A 5A

//Host sends FW Packet (4096 bytes).

00 00 00 00 00 00 00 00 00 00 00 00 00 00 10 00 08 00 00 00 00 00 10 00 00

//Host sends FW Packet Checksum (4 bytes). ([Checksum Code for FW Packet](#))

00 0E 7B 22

//Module responds with **0x69**.

69

//Module responds with **0x5A**.

5A

//Host sends the Total FW Checksum (4 bytes). ([Checksum Code for FW Packet](#))

00 0E 7B 22

//Module responds with **0x5A**.

5A

//Host sends **0x5A/0xA5**. (If this FW is *ag3335_bootloader.bin*, send **0x5A**, otherwise send **0xA5**.)

A5

//Module responds with **0x5A**.

5A

//Host sends **0xB2**.

B2

//Host sends FW Flash Address (*ag3335_bootloader.bin*).

08 00 30 00

//Host sends FW Total Length (*ag3335_bootloader.bin*). (4K alignment: FW total length must be an integer multiple of 4K.)

00 00 50 00

//Host sends FW Packet Length (4 bytes).

```

00 00 10 00
//Module responds with 0x5A5A.
5A 5A
//Host sends FW Packet (4096 bytes).
DF F8 3C D0 72 B6 00 20 03 21 0E 4A 42 F8 04 0B 11 60 0D 4A 0D 4B 00 F0 .....
//Host sends FW Packet Checksum (4 bytes). (Checksum Code for FW Packet)
00 05 AD 0C
//Module responds with 0x69.
69
//Host sends FW Packet (4096 bytes).
D4 30 00 20 10 BD 00 BF C7 28 00 04 4F F4 40 73 A1 F8 BC 30 4F F4 80 73 .....
//Host sends FW Packet Checksum (4 bytes). (Checksum Code for FW Packet)
00 05 67 E0
//Module responds with 0x69.
69
//Host sends FW Packet (4096 bytes).
65 6C 20 70 61 73 73 21 0D 0A 00 72 69 73 69 6E 67 20 66 72 65 71 20 74 .....
//Host sends FW Packet Checksum (4 bytes). (Checksum Code for FW Packet)
00 05 D5 53
//Module responds with 0x69.
69
//Host sends FW Packet (4096 bytes).
70 B5 D0 F8 C8 30 DA 06 04 46 0D 46 01 D4 FF F7 54 FF D4 F8 C8 30 DB 06 .....
//Host sends FW Packet Checksum (4 bytes). (Checksum Code for FW Packet)
00 06 D9 83
//Module responds with 0x69.
69
//Host sends FW Packet (4096 bytes).
06 48 04 E0 06 48 02 E0 06 48 00 E0 06 48 00 EB 81 00 FF F7 B3 BF 6F F0 .....
//Host sends FW Packet Checksum (4 bytes). (Checksum Code for FW Packet)
00 0D 0D 06
//Module responds with 0x69.
69
//Module responds with 0x5A.
5A
//Host sends the total FW Checksum (4 bytes). (Checksum Code for FW Packet)
00 24 D0 C8
//Module responds with 0x5A.
5A
//Host sends 0x5A/0xA5. (If this FW is ag3335_bootloader.bin, send 0x5A, otherwise send 0xA5.)
5A
//Module responds with 0x5A.
5A
//Host sends 0xB2.

```



```
//Module responds with 0x5A.
5A
//Host sends 0xB2.
B2
//Host sends FW Flash Address (gnss_config.bin). See Table 6: Format Length and FW Flash Address \(General Applications\).
08 3D F0 00
//Host sends FW Total Length (gnss_config.bin). (4K alignment: FW total length must be an integer multiple of 4K.)
00 00 10 00
//Host sends FW Packet Length (4 bytes).
00 00 10 00
//Module responds with 0x5A5A.
5A 5A
//Host sends FW Packet (4096 bytes).
01 01 01 FF FF FF FF FF FF FF FF FF FF FF FF FF FF 01 64 00 00 00 FF FF FF .....
//Host sends FW Packet Checksum (4 bytes). (Checksum Code for FW Packet)
00 0E 46 9E
//Module responds with 0x69.
69
//Module responds with 0x5A.
5A
//Host sends the Total FW Checksum (4 bytes). (Checksum Code for FW Packet)
00 0E 46 9E
//Module responds with 0x5A.
5A
//Host sends 0x5A/0xA5. (If this FW is ag3335_bootloader.bin, send 0x5A, otherwise send 0xA5.)
A5
//Module responds with 0x5A.
5A
```

3.2. Download Mode – Firmware Upgrade via I2C Interface

Below is an example of LC29H (BA) firmware upgrading via I2C interface.

```
//Figure 6: Handshake – I2C
//Host keeps sending 0xA0. Reset the module.
A0
A0
A0
...
A0
```

```

A0
A0
A0
//Module responds with 0x665F.
5F 66
//Host sends 0x0A.
0A
//Module responds with 0x66F5.
F5 66
//Host sends 0x50.
50
//Module responds with 0x66AF.
AF 66
//Host sends 0x05.
05
//Module responds with 0x66FA.
FA 66
//Host sends 0xD0.
D0
//Module responds with 0x66D0.
D0 66
//Host sends 0x80000008.
08 00 00 80
//Module responds with 0x6680000008.
08 00 00 80 66
//Host sends 0x00000001.
01 00 00 00
//Module responds with 0x6600000001.
01 00 00 00 66
//Module responds with Brom Status.
00 00 66
//Module responds with HW_CODE.
35 33 66
//Module responds with Brom Status.
00 00 66
//Host sends 0xD0.
D0
//Module responds with 0x66D0.
D0 66
//Host sends 0x8000000C.
0C 00 00 80
//Module responds with 0x668000000C.
0C 00 00 80 66
//Host sends 0x00000001.

```

01 00 00 00

//Module responds with **0x6600000001**.

01 00 00 00 66

//Module responds with Brom Status.

00 00 66

//Module responds with HW_SUBCODE.

80 30 66

//Module responds with Brom Status.

00 00 66

//[Figure 8: Disable Module WDT – I2C](#)

//Host sends **0xD2**.

D2

//Module responds with **0x66D2**.

D2 66

//Host sends **0xA2080000**.

00 00 08 A2

//Module responds with **0x66A2080000**.

00 00 08 A2 66

//Host sends **0x00000001**.

01 00 00 00

//Module responds with **0x6600000001**.

01 00 00 00 66

//Module responds with Brom Status.

01 00 66

//Host sends **0x0010**.

10 00

//Module responds with **0x660010**.

10 00 66

//Module responds with Brom Status.

01 00 66

//[Figure 10: Send DA File to Module – I2C](#)

//Host sends **0xD7**.

D7

//Module responds with **0x66D7**.

D7 66

//Host sends DA Run/Start Address.

00 00 00 04

//Module responds with DA Run/Start Address.

00 00 00 04 66

//Host sends DA Length.

9C 5B 00 00

//Module responds with DA Length.

9C 5B 00 00 66

//Host sends **0x00000000**.

```

00 00 00 00
//Module responds with 0x6600000000.
00 00 00 00 66
//Module responds with Brom Status.
00 00 66
//Host sends DA Data, in a chunk of 1024 bytes.
72 B6 11 49 11 4A 4F F0 00 00 91 42 00 F0 05 80 41 F8 04 0B 91 42 7F F6 .....
//Host sends DA Data, in a chunk of 1024 bytes.
FB AF 0D 49 81 F3 08 88 A1 F1 04 01 0B 4A A1 EB 02 02 4F F0 00 00 91 42 .....
.....
//Host sends the last chunk of DA Data. (length ≤ 1024 bytes)
29 00 10 00 00 00 00 00 21 A0 00 00 22 A0 00 00 1B A0 00 00 15 A2 01 03 .....
//Module responds with Brom Checksum of Received DA (2 bytes). (Checksum Code of DA File)
0F 88 66
//Module responds with Brom Status.
00 00 66
//Figure 12: Jump to DA – I2C
//Host sends 0xD5.
D5
//Module responds with 0x66D5.
D5 66
//Host sends DA Run/Start Address.
00 00 00 04
//Module responds with DA Run/Start Address.
00 00 00 04 66
//Module responds with Brom Status.
00 00 66
//Figure 14: Synchronize with DA and Get DA Report – I2C
//Module responds with 0x66C0.
C0 66
//Host sends 0x3F.
3F
//Module responds with 0x660C.
0C 66
//Host sends 0xF3.
F3
//Module responds with 0x663F.
3F 66
//Host sends 0xC0.
C0
//Module responds with 0x66F3.
F3 66
//Host sends 0x0C.
0C

```

```
//Module responds with 0x665A.
5A 66
//Host sends 0x01
01
//Module responds with 0x6669.
69 66
//Module responds with Flash Manufacturer ID.
EF 00 66
//Module responds with Flash Device ID 1.
60 00 66
//Module responds with Flash Device ID 2.
16 00 66
//Module responds with Flash Start Address.
00 00 00 08 66
//Module responds with Flash Size.
00 00 40 00 66
//Figure 17: Format Flash – I2C
//Host sends 0x05 (Header).
05
//Host sends 0x5A (CMD Type).
5A
//Host sends 0x0A (Length low byte).
0A
//Host sends 0x00 (Length high byte).
00
//Host sends 0x04 (CMD_ID low byte).
04
//Host sends 0x21 (CMD_ID high byte).
21
//Host sends 0x00 (Format Start Address lowest byte).
00
//Host sends 0x00 (Format Start Address second lowest byte).
00
//Host sends 0x00 (Format Start Address second highest byte).
00
//Host sends 0x08 (Format Start Address highest byte).
08
//Host sends 0x00 (Format Size lowest byte).
00
//Host sends 0x00 (Format Size second lowest byte).
00
//Host sends 0x01 (Format Size second highest byte).
01
//Host sends 0x00 (Format Size highest byte).
```

```

00
//Module responds with RACE_DA_FORMAT_FLASH_RESPONSE
05 5B 07 00 04 21 00 00 00 08 66
//Host sends 0x05 (Header).
05
//Host sends 0x5A (CMD Type).
5A
//Host sends 0x0A (Length low byte).
0A
//Host sends 0x00 (Length high byte).
00
//Host sends 0x04 (CMD_ID low byte).
04
//Host sends 0x21 (CMD_ID high byte).
21
//Host sends 0x00 (Format Start Address lowest byte).
00
//Host sends 0x00 (Format Start Address second lowest byte).
00
//Host sends 0x01 (Format Start Address second highest byte).
01
//Host sends 0x08 (Format Start Address highest byte).
08
//Host sends 0x00 (Format Size lowest byte).
00
//Host sends 0x00 (Format Size second lowest byte).
00
//Host sends 0x01 (Format Size second highest byte).
01
//Host sends 0x00 (Format Size highest byte).
00
//Module responds with RACE_DA_FORMAT_FLASH_RESPONSE
05 5B 07 00 04 21 00 00 00 01 08 66
.....
//Host sends 0x05 (Header).
05
//Host sends 0x5A (CMD Type).
5A
//Host sends 0x0A (Length low byte).
0A
//Host sends 0x00 (Length high byte).
00
//Host sends 0x04 (CMD_ID low byte).
04

```

```
//Host sends 0x21 (CMD_ID high byte).
21
//Host sends 0x00 (Format Start Address lowest byte).
00
//Host sends 0x00 (Format Start Address second lowest byte).
00
//Host sends 0x3D (Format Start Address second highest byte).
3D
//Host sends 0x08 (Format Start Address highest byte).
08
//Host sends 0x00 (Format Size lowest byte).
00
//Host sends 0x00 (Format Size second lowest byte).
00
//Host sends 0x01 (Format Size second highest byte).
01
//Host sends 0x00 (Format Size highest byte).
00
//Module responds with RACE_DA_FORMAT_FLASH_RESPONSE
05 5B 07 00 04 21 00 00 00 3D 08 66
//Figure 19: Send FW File to Module – I2C
//Host sends 0x05 (Header).
05
//Host sends 0x5A (CMD Type).
5A
//Host sends 0x0C (Length low byte).
0C
//Host sends 0x10 (Length high byte).
10
//Host sends 0x00 (CMD_ID low byte).
00
//Host sends 0x21 (CMD_ID high byte).
21
//Host sends 0x00 (partition_table.bin Data Start Address lowest byte).
00
//Host sends 0x00 (partition_table.bin Data Start Address second lowest byte).
00
//Host sends 0x00 (partition_table.bin Data Start Address second highest byte).
00
//Host sends 0x08 (partition_table.bin Data Start Address highest byte).
08
//Host sends 0x00 (Data length low byte).
00
//Host sends 0x10 (Data length high byte).
```

```

10
//Host sends 0x22 (Checksum of Data lowest byte Checksum Code for FW Packet).
22
//Host sends 0x7B (Checksum of Data second lowest byte Checksum Code for FW Packet).
7B
//Host sends 0x0E (Checksum of Data second highest byte Checksum Code for FW Packet).
0E
//Host sends 0x00 (Checksum of Data highest byte Checksum Code for FW Packet).
00
//Host sends Data (4096 bytes).
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 10 00 08 00 00 00 00 00 10 00 00 .....
//Module responds with RACE_DA_WRITE_DATA_RESPONSE
05 5B 07 00 00 21 00 00 00 00 08 66
//Host sends 0x05 (Header).
05
//Host sends 0x5A (CMD Type).
5A
//Host sends 0x0C (Length low byte).
0C
//Host sends 0x10 (Length high byte).
10
//Host sends 0x00 (CMD_ID low byte).
00
//Host sends 0x21 (CMD_ID high byte).
21
//Host sends 0x00 (ag3335_bootloader.bin Data Start Address lowest byte).
00
//Host sends 0x30 (ag3335_bootloader.bin Data Start Address second lowest byte).
30
//Host sends 0x00 (ag3335_bootloader.bin Data Start Address second highest byte).
00
//Host sends 0x08 (ag3335_bootloader.bin Data Start Address highest byte).
08
//Host sends 0x00 (Data Length low byte).
00
//Host sends 0x10 (Data Length high byte).
10
//Host sends 0xDA (Checksum of FW Packet lowest byte Checksum Code for FW Packet).
DA
//Host sends 0xAB (Checksum of FW Packet second lowest byte Checksum Code for FW Packet).
AB
//Host sends 0x05 (Checksum of FW Packet second highest byte Checksum Code for FW Packet).
05
//Host sends 0x00 (Checksum of FW Packet highest byte Checksum Code for FW Packet).

```



```

00
//Host sends Data (4096 bytes).
F8 DF D0 3C B6 72 20 00 21 03 4A 0E F8 42 0B 04 11 60 0D 4A 0D 4B 00 F0 .....
//Module responds with RACE_DA_WRITE_DATA_RESPONSE
05 5B 07 00 00 21 00 00 30 00 08 66
.....
//Host sends 0x05 (Header).
05
//Host sends 0x5A (CMD Type).
5A
//Host sends 0x0C (Length low byte).
0C
//Host sends 0x10 (Length high byte).
10
//Host sends 0x00 (CMD_ID low byte).
00
//Host sends 0x21 (CMD_ID high byte).
21
//Host sends 0x00 (ag3335_bootloader.bin Data Start Address lowest byte).
00
//Host sends 0x70 (ag3335_bootloader.bin Data Start Address second lowest byte).
70
//Host sends 0x00 (ag3335_bootloader.bin Data Start Address second highest byte).
00
//Host sends 0x08 (ag3335_bootloader.bin Data Start Address highest byte).
08
//Host sends 0x00 (Data Length low byte).
00
//Host sends 0x10 (Data Length high byte).
10
//Host sends 0x9C (Checksum of Data lowest byte Checksum Code for FW Packet).
9C
//Host sends 0xD5 (Checksum of Data second lowest byte Checksum Code for FW Packet).
D5
//Host sends 0x06 (Checksum of Data second highest byte Checksum Code for FW Packet).
06
//Host sends 0x00 (Checksum of Data highest byte Checksum Code for FW Packet).
00
//Host sends Data (4096 bytes).
C6 00 59 78 01 F0 C2 F8 07 4B B4 F8 C6 00 01 22 1A 70 D4 F8 B0 30 2A 46 .....
//Module responds with RACE_DA_WRITE_DATA_RESPONSE
05 5B 07 00 00 21 00 00 70 00 08 66
//Host sends 0x05 (Header).
05

```

```
//Host sends 0x5A (CMD Type).
5A
//Host sends 0x0C (Length low byte).
0C
//Host sends 0x10 (Length high byte).
10
//Host sends 0x00 (CMD_ID low byte).
00
//Host sends 0x21(CMD_ID high byte).
21
//Host sends 0x00 (L29HBANRxxAxxS.bin Data Start Address lowest byte).
00
//Host sends 0x30 (L29HBANRxxAxxS.bin Data Start Address second lowest byte).
30
//Host sends 0x01 (L29HBANRxxAxxS.bin Data Start Address second highest byte).
01
//Host sends 0x08 (L29HBANRxxAxxS.bin Data Start Address highest byte).
08
//Host sends 0x00 (Data Length low byte).
00
//Host sends 0x10 (Data Length high byte).
10
//Host sends 0xB5 (Checksum of Data lowest byte Checksum Code for FW Packet).
B5
//Host sends 0xA2 (Checksum of Data second lowest byte Checksum Code for FW Packet).
A2
//Host sends 0x06 (Checksum of Data second highest byte Checksum Code for FW Packet).
06
//Host sends 0x00 (Checksum of Data highest byte Checksum Code for FW Packet).
00
//Host sends Data (4096 bytes).
DF F8 A0 D0 72 B6 28 48 80 47 28 48 28 49 01 60 28 48 29 49 01 60 29 26 .....
//Module responds with RACE_DA_WRITE_DATA_RESPONSE
05 5B 07 00 00 21 00 00 30 01 08 66
...
//Host sends 0x05 (Header).
05
//Host sends 0x5A (CMD Type).
5A
//Host sends 0x0C (Length low byte).
0C
//Host sends 0x10 (Length high byte).
10
//Host sends 0x00 (CMD_ID low byte).
```

```

00
//Host sends 0x21(CMD_ID high byte).
21
//Host sends 0x00 (L29HBANRxxAxxS.bin Data Start Address lowest byte).
00
//Host sends 0x90 (L29HBANRxxAxxS.bin Data Start Address second lowest byte).
90
//Host sends 0x1F (L29HBANRxxAxxS.bin Data Start Address second highest byte).
1F
//Host sends 0x08 (L29HBANRxxAxxS.bin Data Start Address highest byte).
08
//Host sends 0x00 (Data Length low byte).
00
//Host sends 0x10 (Data Length high byte).
10
//Host sends 0x4E (Checksum of Data lowest byte Checksum Code for FW Packet).
4E
//Host sends 0x7B (Checksum of Data second lowest byte Checksum Code for FW Packet).
7B
//Host sends 0x0E (Checksum of Data second highest byte Checksum Code for FW Packet).
0E
//Host sends 0x00 (Checksum of Data highest byte Checksum Code for FW Packet).
00
//Host sends Data (4096 bytes).
48 D0 56 56 6E B3 26 A6 30 6F 23 32 8F FE 1E F3 60 40 47 4D 69 22 27 81 .....
//Module responds with RACE_DA_WRITE_DATA_RESPONSE
05 5B 07 00 00 21 00 00 90 1F 08 66
//Host sends 0x05 (Header).
05
//Host sends 0x5A (CMD Type).
5A
//Host sends 0x0C (Length low byte).
0C
//Host sends 0x10 (Length high byte).
10
//Host sends 0x00 (CMD_ID low byte).
00
//Host sends 0x21(CMD_ID high byte).
21
//Host sends 0x00 (gnss_config.bin Data Start Address lowest byte).
00
//Host sends 0xF0 (gnss_config.bin Data Start Address second lowest byte).
F0
//Host sends 0x3D (gnss_config.bin Data Start Address second highest byte).

```

```

3D
//Host sends 0x08 (gnss_config.bin Data Start Address highest byte).
08
//Host sends 0x00 (Data Length low byte).
00
//Host sends 0x10 (Data Length high byte).
10
//Host sends 0xEB (Checksum of Data lowest byte).
EB
//Host sends 0x32 (Checksum of Data second lowest byte).
32
//Host sends 0x0E (Checksum of Data second highest byte).
0E
//Host sends 0x00 (Checksum of Data highest byte).
00
//Host sends Data (4096 bytes).
01 01 01 01 FF FF FF FF FF FF FF FF FF FF FF FF 01 64 00 00 00 FF FF .....
//Module responds with RACE_DA_WRITE_DATA_RESPONSE
05 5B 07 00 00 21 00 00 F0 3D 08 66

```

3.3. Upgrade Mode – Firmware Upgrade via SPI

LC29H (AI) is used as an example to show how to upgrade firmware via SPI in Upgrade mode. The **XX** below represents dummy data.

```

//Handshake with module
//Host sends CFG_WR_CMD (0x0C) and SPIS_ADDRESS_ID with 0x00000003.
0C 00 00 AA 55 03 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends WR_CMD (0x0E) and HANDSHAKE_WORD1.
0E 09 13 4C 51
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.
XX 23
//Host sends CFG_RD_CMD (0x0A) and SPIS_ADDRESS_ID with 0x00000003.
0A 00 00 AA 55 03 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX

```

```
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends RD_CMD (0x81) with 4 bytes of dummy data.
81 XX XX XX XX
//Module responds with RSP_WORD1.
XX 4D 3A FC AA
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.
XX 23
//Host sends CFG_WR_CMD (0x0C) and SPIS_ADDRESS_ID with 0x00000003.
0C 00 00 AA 55 03 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends WR_CMD (0x0E) and HANDSHAKE_WORD2.
0E 04 A5 03 12
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 00
//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.
XX 23
//Host sends CFG_RD_CMD (0x0A) and SPIS_ADDRESS_ID with 0x00000003.
0A 00 00 AA 55 03 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends RD_CMD (0x81) with 4 bytes of dummy data.
81 XX XX XX XX
//Module responds with RSP_WORD2.
XX A0 5B FD 55
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.
XX 23

//Send firmware address (partition_table.bin)
//Host sends CFG_WR_CMD (0x0C) and SPIS_ADDRESS_ID with 0x0000000D.
0C 00 00 AA 55 0D 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
```

//Host sends **WR_CMD** (0x0E) and the firmware address frame. See [Table 21: Firmware Address Frame from Host](#).

0E AA 02 01 00 04 08 00 00 00 B1 44 5D 8F 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Host sends **CFG_RD_CMD** (0x0A) and SPIS_ADDRESS_ID with **0x0000000D**.

0A 00 00 AA 55 0D 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **RD_CMD** (0x81) with 14 bytes dummy data.

81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX

//Module responds to firmware address frame. see [Table 22: Firmware Address Response Frame from Module](#).

XX AA 02 00 00 04 02 01 00 00 79 4C DC 68 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Send firmware information (*partition_table.bin*)

//Host sends **CFG_WR_CMD** (0x0C) and SPIS_ADDRESS_ID with **0x00000019**.

0C 00 00 AA 55 19 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **WR_CMD** (0x0E) and the firmware information frame. See [Table 23: Firmware Information Frame from Host](#).

0E AA 02 02 00 10 00 00 10 00 46 69 6D 42 08 00 00 00 01 00 00 00 8B AF 2C DD 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Host sends **CFG_RD_CMD** (0x0A) and SPIS_ADDRESS_ID with **0x0000000D**.

0A 00 00 AA 55 0D 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **RD_CMD** (0x81) with 14 bytes dummy data.

81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX

//Module responds to firmware information frame. See [Table 25: Firmware Information Response Frame from Module](#).

XX AA 02 00 00 04 02 02 00 00 7B 0A 62 31 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Send start firmware upgrade command

//Host sends **CFG_WR_CMD** (0x0C) and SPIS_ADDRESS_ID with **0x00000009**.

0C 00 00 AA 55 09 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **WR_CMD** (0x0E) and start firmware upgrade frame. See [Table 26: Start Firmware Upgrade Frame from Host](#).

0E AA 02 03 00 00 89 0B A9 CE 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Host sends **CFG_RD_CMD** (0x0A) and SPIS_ADDRESS_ID with **0x0000000D**.

0A 00 00 AA 55 0D 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **RD_CMD** (0x81) with 14 bytes dummy data.

81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX

//Module responds to start firmware upgrade frame. See [Table 27: Start Firmware Upgrade Response Frame from Module](#).

XX AA 02 00 00 04 02 03 00 00 7A C8 08 06 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Format flash (*partition_table.bin*)

//Host sends **CFG_WR_CMD** (0x0C) and SPIS_ADDRESS_ID with **0x00000011**.

0C 00 00 AA 55 11 00 00 00

//Host sends **RS_CMD** and **XX** to read the current SPI status.

06 XX

```
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends WR_CMD (0x0E) and the format flash frame. See Table 28: Format Flash Frame from Host.
0E AA 02 21 00 08 08 00 00 00 00 10 00 F2 03 A3 1B 55
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.
XX 23
//Host sends CFG_RD_CMD (0x0A) and SPIS_ADDRESS_ID with 0x0000000E.
0A 00 00 AA 55 0E 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status. The correct status will not be
returned promptly and the host needs to read in a loop for timeout processing.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends RD_CMD (0x81) with 15 bytes dummy data.
81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX
//Module responds with format progress. See Table 30: Format Progress Frame from Module.
XX AA 02 00 00 05 02 21 64 00 00 B5 A2 55 5C 55
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.
XX 23
//Host sends CFG_WR_CMD (0x0C) and SPIS_ADDRESS_ID with 0x0000000E.
0C 00 00 AA 55 0E 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends WR_CMD (0x0E) and the format progress response frame (see Table 31: Format Progress
Response Frame from Host).
0E AA 02 21 00 05 00 00 64 00 00 E2 30 AC EF 55
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.
XX 23
//Host sends CFG_RD_CMD (0x0A) and SPIS_ADDRESS_ID with 0x0000000D.
0A 00 00 AA 55 0D 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends RD_CMD (0x81) with 14 bytes dummy data.
81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX
```


//Module responds to format flash frame. See [Table 32: Format Flash Response Frame from Module](#).

XX AA 02 00 00 04 02 21 00 00 41 01 9A 88 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Send FW file to module (*partition_table.bin*)

//Host sends **CFG_WR_CMD** (0x0C) and SPIS_ADDRESS_ID with **0x0000100D**.

0C 00 00 AA 55 0D 10 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **WR_CMD** (0x0E) and the FW file frame. See [Table 33: FW File Frame from Host](#).

0E AA 02 04 10 04 00 00 00 00 00 00 00 00 ... FF FF CF 8B 0C E9 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Host sends **CFG_RD_CMD** (0x0A) and SPIS_ADDRESS_ID with **0x0000000D**.

0A 00 00 AA 55 0D 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **RD_CMD** (0x81) with 14 bytes dummy data.

81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX

//Module responds to FW file frame. See [Table 34: FW File Response Frame from Module](#).

XX AA 02 00 00 04 02 04 00 00 7F 87 1E 83 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Send firmware address (*LC29HAINRxxAxxS.bin*)

//Host sends **CFG_WR_CMD** (0x0C) and SPIS_ADDRESS_ID with **0x0000000D**.

0C 00 00 AA 55 0D 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **WR_CMD** (0x0E) and the firmware address frame. See [Table 21: Firmware Address Frame from Host](#).

0E AA 02 01 00 04 08 01 30 00 6F C0 01 4B 55

//Host sends **RS_CMD** and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Host sends **CFG_RD_CMD** (0x0A) and SPIS_ADDRESS_ID with **0x0000000D**.

0A 00 00 AA 55 0D 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **RD_CMD** (0x81) with 14 bytes dummy data.

81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX

//Module responds to firmware address frame. See [Table 22: Firmware Address Response Frame from Module](#).

XX AA 02 00 00 04 02 01 00 00 79 4C DC 68 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Send firmware information (LC29HAINRxxAxxS.bin)

//Host sends **CFG_WR_CMD** (0x0C) and SPIS_ADDRESS_ID with **0x00000019**.

0C 00 00 AA 55 19 00 00 00

//Host sends **RS_CMD** and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **WR_CMD** (0x0E) and the firmware information frame. See [Table 23: Firmware Information Frame from Host](#).

0E AA 02 02 00 10 00 15 31 1B 6B 74 3D 07 08 01 30 00 01 00 00 00 93 0F 12 0D 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Host sends **CFG_RD_CMD** (0x0A) and SPIS_ADDRESS_ID with **0x0000000D**.

0A 00 00 AA 55 0D 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **RD_CMD** (0x81) with 14 bytes dummy data.

81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX

//Module responds to firmware information frame. See [Table 25: Firmware Information Response Frame](#)

[from Module.](#)

XX AA 02 00 00 04 02 02 00 00 7B 0A 62 31 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Send start firmware upgrade command

//Host sends **CFG_WR_CMD** (0x0C) and SPIS_ADDRESS_ID with **0x00000009**.

0C 00 00 AA 55 09 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **WR_CMD** (0x0E) and the start firmware upgrade frame. See [Table 26: Start Firmware Upgrade Frame from Host](#).

0E AA 02 03 00 00 89 0B A9 CE 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Host sends **CFG_RD_CMD** (0x0A) and SPIS_ADDRESS_ID with **0x0000000D**.

0A 00 00 AA 55 0D 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **RD_CMD** (0x81) with 14 bytes dummy data.

81 XX XX XX XX XX XX XX XX XX XX XX XX XX

//Module responds to start firmware upgrade frame. See [Table 27: Start Firmware Upgrade Response Frame from Module](#).

XX AA 02 00 00 04 02 03 00 00 7A C8 08 06 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Format Flash (LC29HAINRxxAxxS.bin)

//Host sends **CFG_WR_CMD** (0x0C) and SPIS_ADDRESS_ID with **0x00000011**.

0C 00 00 AA 55 11 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

```
//Host sends WR_CMD (0x0E) and the format flash frame. See Table 28: Format Flash Frame from Host.
0E AA 02 21 00 08 08 01 30 00 00 3C C0 00 FC 47 76 E9 55
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.
XX 23
//Host sends CFG_RD_CMD (0x0A) and SPIS_ADDRESS_ID with 0x0000000E.
0A 00 00 AA 55 0E 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status. The correct status will not be
returned promptly and the host needs to read in a loop for timeout processing.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit2 = 1) status flag.
XX 07
//Host sends RD_CMD (0x81) with 15 bytes dummy data.
81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX
//Module responds with format progress. See Table 30: Format Progress Frame from Module.
XX AA 02 00 00 05 02 21 01 00 00 FB BE 5C 97 55
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.
XX 23
//Host sends CFG_WR_CMD (0x0C) and SPIS_ADDRESS_ID with 0x0000000E.
0C 00 00 AA 55 0E 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends WR_CMD (0x0E) and the format progress response frame. See Table 31: Format Progress
Response Frame from Host.
0E AA 02 21 00 05 00 00 01 00 00 AC 2C A5 64 55
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.
XX 23
//Host sends CFG_WR_CMD (0x0C) and SPIS_ADDRESS_ID with 0x0000000E.
0C 00 00 AA 55 0E 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends RD_CMD (0x81) with 15 bytes dummy data.
81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX
//Module responds with format progress. See Table 30: Format Progress Frame from Module.
XX AA 02 00 00 05 02 21 03 00 00 FB BE 5C 97 55
```

```
//Host sends CFG_WR_CMD (0x0C) and SPIS_ADDRESS_ID with 0x0000000E.
0C 00 00 AA 55 0E 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends WR_CMD (0x0E) and the format progress response frame. See Table 31: Format Progress Response Frame from Host.
0E AA 02 21 00 05 00 00 03 00 00 AF A8 71 0A 55
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.
XX 23
...
//Host sends CFG_WR_CMD (0x0C) and SPIS_ADDRESS_ID with 0x0000000E.
0C 00 00 AA 55 0E 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends RD_CMD (0x81) with 15 bytes dummy data.
81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX
//Module responds with format progress. See Table 30: Format Progress Frame from Module.
XX AA 02 00 00 05 02 21 64 00 00 B5 A2 55 5C 55
//Host sends CFG_WR_CMD (0x0C) and SPIS_ADDRESS_ID with 0x0000000E.
0C 00 00 AA 55 0E 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends WR_CMD (0x0E) and the format progress response frame. See Table 31: Format Progress Response Frame from Host.
0E AA 02 21 00 05 00 00 64 00 00 E2 30 AC EF 55
//Host sends RS_CMD and XX to read the current SPI status.
06 XX
//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.
XX 23
//Host sends CFG_RD_CMD (0x0A) and SPIS_ADDRESS_ID with 0x0000000D.
0A 00 00 AA 55 0D 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends RD_CMD (0x81) with 14 bytes dummy data.
```

81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX

//Module responds to format flash frame. See [Table 32: Format Flash Response Frame from Module](#).

XX AA 02 00 00 04 02 21 00 00 41 01 9A 88 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Send FW file to module (*LC29HAINRxxAxxS.bin*)

//Host sends **CFG_WR_CMD** (0x0C) and SPIS_ADDRESS_ID with **0x0000100D**.

0C 00 00 AA 55 0D 10 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **WR_CMD** (0x0E) and the FW file frame. See [Table 33: FW File Frame from Host](#).

0E AA 02 04 10 04 00 00 00 00 DF F8 B4 ... 0A E7 3B AA 6A FA 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Host sends **CFG_RD_CMD** (0x0A) and SPIS_ADDRESS_ID with **0x0000000D**.

0A 00 00 AA 55 0D 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **RD_CMD** (0x81) with 14 bytes dummy data.

81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX

//Module responds to FW file frame. See [Table 34: FW File Response Frame from Module](#).

XX AA 02 00 00 04 02 04 00 00 7F 87 1E 83 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

...

//Host sends **CFG_WR_CMD** (0x0C) and SPIS_ADDRESS_ID with **0x0000100D**.

0C 00 00 AA 55 0D 10 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **WR_CMD** (0x0E) and the FW file frame. See [Table 33: FW File Frame from Host](#).

0E AA 02 04 10 04 00 00 01 53 FF 04 FF ... FF FF 33 65 39 CD 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Host sends **CFG_RD_CMD** (0x0A) and SPIS_ADDRESS_ID with **0x0000000D**.

0A 00 00 AA 55 0D 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **RD_CMD** (0x81) with 14 bytes dummy data.

81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX

//Module responds to FW file frame. See [Table 34: FW File Response Frame from Module](#).

XX AA 02 00 00 04 02 04 00 00 7F 87 1E 83 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Send firmware address (*gnss_config.bin*)

//Host sends **CFG_WR_CMD** (0x0C) and SPIS_ADDRESS_ID with **0x0000000D**.

0C 00 00 AA 55 0D 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **WR_CMD** (0x0E) and the firmware address frame. See [Table 21: Firmware Address Frame from Host](#).

0E AA 02 01 00 04 08 3D F0 00 89 4B CA F1 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Host sends **CFG_RD_CMD**(0x0A) and SPIS_ADDRESS_ID with **0x0000000D**.

0A 00 00 AA 55 0D 00 00 00

//Host sends **RS_CMD** and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **RD_CMD** (0x81) with 14 bytes dummy data.

81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX

//Module responds to firmware address frame. See [Table 22: Firmware Address Response Frame from Module](#).

XX AA 02 00 00 04 02 01 00 00 79 4C DC 68 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Send firmware information(*gnss_config.bin*)

//Host sends **CFG_WR_CMD**(0x0C) and SPIS_ADDRESS_ID with **0x00000019**.

0C 00 00 AA 55 19 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **WR_CMD** (0x0E) and the firmware information frame. See [Table 23: Firmware Information Frame from Host](#).

0E AA 02 02 00 10 00 00 10 00 3B C6 EC D9 08 3D F0 00 01 00 00 00 CE EE 81 1E 55

//Host sends **RS_CMD** and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Host sends **CFG_RD_CMD** (0x0A) and SPIS_ADDRESS_ID with **0x0000000D**.

0A 00 00 AA 55 0D 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **RD_CMD** (0x81) with 14 bytes dummy data.

81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX

//Module responds to firmware information frame. See [Table 25: Firmware Information Response Frame from Module](#).

XX AA 02 00 00 04 02 02 00 00 7B 0A 62 31 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Send start firmware upgrade command

//Host sends **CFG_WR_CMD** (0x0C) and SPIS_ADDRESS_ID with **0x00000009**.

0C 00 00 AA 55 09 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **WR_CMD** (0x0E) and the start firmware upgrade frame. See [Table 26: Start Firmware Upgrade Frame from Host](#).

0E AA 02 03 00 00 89 0B A9 CE 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Host sends **CFG_RD_CMD**(0x0A) and SPIS_ADDRESS_ID with **0x0000000D**.

0A 00 00 AA 55 0D 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **RD_CMD** (0x81) with 14 bytes dummy data.

81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX

//Module responds to start firmware upgrade frame. See [Table 27: Start Firmware Upgrade Response Frame from Module](#).

XX AA 02 00 00 04 02 03 00 00 7A C8 08 06 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Format Flash (*gnss_config.bin*).

//Host sends **CFG_WR_CMD**(0x0C) and SPIS_ADDRESS_ID with **0x00000011**.

0C 00 00 AA 55 11 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **WR_CMD** (0x0E) and the format flash frame. See [Table 28: Format Flash Frame from Host](#).

0E AA 02 21 00 08 08 3D F0 00 00 00 10 00 2E 25 7E 90 55

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status.

06 XX

//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.

XX 23

//Host sends **CFG_RD_CMD** (0x0A) and SPIS_ADDRESS_ID with **0x0000000E**.

0A 00 00 AA 55 0E 00 00 00

//Host sends **RS_CMD** (0x06) and **XX** to read the current SPI status. The correct status will not be returned promptly and the host needs to read in a loop for timeout processing.

06 XX

//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.

XX 07

//Host sends **RD_CMD** (0x81) with 15 bytes dummy data.

81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX

//Module responds with format progress. See [Table 30: Format Progress Frame from Module](#).

```

XX AA 02 00 00 05 02 21 64 00 00 B5 A2 55 5C 55
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.
XX 23
//Host sends CFG_WR_CMD (0x0C) and SPIS_ADDRESS_ID with 0x0000000E.
0C 00 00 AA 55 0E 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends WR_CMD (0x0E) and the format progress response frame. See Table 31: Format Progress Response Frame from Host.
0E AA 02 21 00 05 00 00 64 00 00 E2 30 AC EF 55
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.
XX 23
//Host sends CFG_RD_CMD(0x0A) and SPIS_ADDRESS_ID with 0x0000000D.
0A 00 00 AA 55 0D 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends RD_CMD (0x81) with 14 bytes dummy data.
81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX
//Module responds to format flash frame. See Table 32: Format Flash Response Frame from Module.
XX AA 02 00 00 04 02 21 00 00 41 01 9A 88 55
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.
XX 23

//Send FW file to module (gnss_config.bin)
//Host sends CFG_WR_CMD(0x0C) and SPIS_ADDRESS_ID with 0x0000100D.
0C 00 00 AA 55 0D 10 00 00
//Host sends RS_CMD and XX to read the current SPI status.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends WR_CMD (0x0E) and the FW file frame. See Table 33: FW File Frame from Host.
0E AA 02 04 10 04 00 00 00 00 FF FF FF ... FF FF 3B C6 EC D9 55
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX

```

```
//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.
XX 23
//Host sends CFG_RD_CMD (0x0A) and SPIS_ADDRESS_ID with 0x0000000D.
0A 00 00 AA 55 0D 00 00 00
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_TXRX_FIFO_RDY (Bit 2 = 1) status flag.
XX 07
//Host sends RD_CMD (0x81) with 14 bytes dummy data.
81 XX XX XX XX XX XX XX XX XX XX XX XX XX XX
//Module responds to FW file frame. See Table 34: FW File Response Frame from Module.
XX AA 02 00 00 04 02 04 00 00 7F 87 1E 83 55
//Host sends RS_CMD (0x06) and XX to read the current SPI status.
06 XX
//Module responds with data including STA_RDWR_FINISH (Bit 5 = 1) status flag.
XX 23
```

3.4. Upgrade Mode – Firmware Upgrade via I2C Interface

LC79H (AL) is used as an example to show how to upgrade firmware via I2C interface in Upgrade mode.

```
//Handshake with module
//Host sends HANDSHAKE_WORD1.
09 13 4C 51
//Host sends I2C_DATA_Len_reg.
AA 51 00 08
//Host reads data length.
04 00 00 00
//Host reads RSP_WORD1.
4D 3A FC AA
//Host sends HANDSHAKE_WORD2.
04 A5 03 12
//Host sends I2C_DATA_Len_reg.
AA 51 00 08
//Host reads data length
04 00 00 00
//Host reads RSP_WORD2.
A0 5B FD 55

//Send firmware address (partition_table.bin)
//Host sends the firmware address frame. See Table 21: Firmware Address Frame from Host.
AA 02 01 00 04 08 00 00 00 B1 44 5D 8F 55
```

//Host sends I2C_DATA_Len_reg.

AA 51 00 08

//Host reads data length.

0E 00 00 00

//Host reads module's response to firmware address frame. See [Table 22: Firmware Address Response Frame from Module](#).

AA 02 00 00 04 02 01 00 00 79 4C DC 68 55

//Send firmware information (*partition_table.bin*)

//Host sends firmware information frame. See [Table 23: Firmware Information Frame from Host](#).

AA 02 02 00 10 00 00 10 00 9A 65 2F 76 08 00 00 00 01 00 00 00 31 91 80 34 55

//Host sends I2C_DATA_Len_reg.

AA 51 00 08

//Host reads data length.

0E 00 00 00

//Host reads module's response to firmware information frame. See [Table 25: Firmware Information Response Frame from Module](#).

AA 02 00 00 04 02 02 00 00 7B 0A 62 31 55

//Sen start firmware upgrade command

//Host sends the start firmware upgrade frame. See [Table 26: Start Firmware Upgrade Frame from Host](#).

AA 02 03 00 00 89 0B A9 CE 55

//Host sends I2C_DATA_Len_reg.

AA 51 00 08

//Host reads data length.

0E 00 00 00

//Host reads module's response to start firmware upgrade frame. See [Table 27: Start Firmware Upgrade Response Frame from Module](#).

AA 02 00 00 04 02 03 00 00 7A C8 08 06 55

//Format flash (*partition_table.bin*).

//Host sends the format flash frame. See [Table 28: Format Flash Frame from Host](#).

AA 02 21 00 08 08 00 00 00 00 10 00 F2 03 A3 1B 55

//Host sends I2C_DATA_Len_reg.

AA 51 00 08

//Host reads data length.

0F 00 00 00

//Host reads module's format progress. See [Table 30: Format Progress Frame from Module](#).

AA 02 00 00 05 02 21 64 00 00 B5 A2 55 5C 55

//Host sends the format progress response frame. See [Table 31: Format Progress Response Frame from Host](#).

AA 02 21 00 05 00 00 64 00 00 E2 30 AC AF 55

//Host sends I2C_DATA_Len_reg.

AA 51 00 08

//Host reads data length.

0E 00 00 00

//Host reads module's response to format flash frame. See [Table 32: Format Flash Response Frame from Module](#).

AA 02 00 00 04 02 21 00 00 41 01 9A 88 55

//Send FW file to module (*partition_table.bin*)

//Host sends the FW file frame. See [Table 33: FW File Frame from Host](#). (The frame size is 256 bytes.)

AA 02 04 10 04 00 00 00 00 00 00 ... FF FF CF 8B 0C E9 55

//Host sends I2C_DATA_Len_reg.

AA 51 00 08

//Host reads data length.

0E 00 00 00

//Module responds to FW file frame. See [Table 34: FW File Response Frame from Module](#).

AA 02 00 00 04 02 04 00 00 7F 87 1E 83 55

//Send firmware address (*LC79HALNRxxAxxS.bin*)

//Host sends the firmware address frame. See [Table 21: Firmware Address Frame from Host](#).

AA 02 04 10 04 00 00 00 00 00 00 ... FF FF CF 8B 0C E9 55

//Host sends I2C_DATA_Len_reg.

AA 51 00 08

//Host reads data length.

0E 00 00 00

//Host reads module's response to firmware address frame. See [Table 22: Firmware Address Response Frame from Module](#).

AA 02 00 00 04 02 04 00 00 7F 87 1E 83 55

//Send firmware information (*LC79HALNRxxAxxS.bin*)

//Host sends the firmware information frame. See [Table 23: Firmware Information Frame from Host](#).

AA 02 02 00 10 00 17 95 AB C1 85 EF 28 08 01 30 00 01 00 00 00 17 01 85 56 55

//Host sends I2C_DATA_Len_reg.

AA 51 00 08

//Host reads data length.

0E 00 00 00

//Host reads module's response to firmware information frame. See [Table 25: Firmware Information Response Frame from Module](#).

AA 02 00 00 04 02 02 00 00 7B 0A 62 31 55

//Send start firmware upgrade command

//Host sends the start firmware upgrade frame. See [Table 26: Start Firmware Upgrade Frame from Host](#).

AA 02 03 00 00 89 0B A9 CE 55

//Host sends I2C_DATA_Len_reg.

AA 51 00 08

//Host reads data length.

0E 00 00 00

//Host reads module's response to start firmware upgrade frame. See [Table 27: Start Firmware Upgrade Response Frame from Module](#).

AA 02 00 00 04 02 03 00 00 7A C8 08 06 55

//Format Flash (LC79HALNRxxAxxS.bin)

//Host sends the format flash frame. See [Table 28: Format Flash Frame from Host](#).

AA 02 21 00 08 08 01 30 00 00 3C C0 00 FC 47 76 E9 55

//Host sends I2C_DATA_Len_reg.

AA 51 00 08

//Host reads data length.

0F 00 00 00

//Host reads module's format progress. See [Table 30: Format Progress Frame from Module](#).

AA 02 00 00 05 02 21 01 00 00 FB BE 5C 97 55

//Host sends the format progress response frame. See [Table 31: Format Progress Response Frame from Host](#).

AA 02 21 00 05 00 00 01 00 00 AC 2C A5 64 55

//Host sends I2C_DATA_Len_reg.

AA 51 00 08

//Host reads data length.

0F 00 00 00

//Host reads module's format progress. See [Table 30: Format Progress Frame from Module](#).

AA 02 00 00 05 02 21 02 00 00 F9 F8 E2 CE 55

//Host sends the format progress response frame. See [Table 31: Format Progress Response Frame from Host](#).

AA 02 21 00 05 00 00 02 00 00 AE 6A 1B 3D 55

...

//Host sends I2C_DATA_Len_reg.

AA 51 00 08

//Host reads data length.

0F 00 00 00

//Host reads module's format progress. See [Table 30: Format Progress Frame from Module](#).

AA 02 00 00 05 02 21 64 00 00 B5 A2 55 5C 55

//Host sends the format progress response frame. See [Table 31: Format Progress Response Frame from Host](#).

AA 02 21 00 05 00 00 64 00 00 E2 30 AC AF 55

//Host sends I2C_DATA_Len_reg.

AA 51 00 08

//Host reads data length.

0F 00 00 00

//Host reads module's response to format flash frame. See [Table 32: Format Flash Response Frame from Module](#).

AA 02 00 00 04 02 21 00 00 41 01 9A 88 55

```
//Send FW file to module (LC79HALNRxxAxxS.bin)
//Host sends the FW file frame. See Table 33: FW File Frame from Host. (The frame size is 256 bytes.)
AA 02 04 10 04 00 00 00 00 DF F8 00 ... 40 F2 F3 80 02 2D 9D 3E 55
//Host sends I2C_DATA_Len_reg.
AA 51 00 08
//Host reads data length.
0E 00 00 00
//Module responds to FW file frame. See Table 34: FW File Response Frame from Module.
AA 02 00 00 04 02 04 00 00 7F 87 1E 83 55
//Host sends the FW file frame. See Table 33: FW File Frame from Host. (The frame size is 256 bytes.)
AA 02 04 10 04 00 00 00 01 02 3A 26 ... 3B F1 D4 F8 EA 3D CE F8 55
//Host sends I2C_DATA_Len_reg.
AA 51 00 08
//Host reads data length.
0E 00 00 00
//Module responds to FW file frame. See Table 34: FW File Response Frame from Module.
AA 02 00 00 04 02 04 00 00 7F 87 1E 83 55
...
//Host sends the FW file frame. See Table 33: FW File Frame from Host. (The frame size is 256 bytes.)
AA 02 04 10 04 00 00 01 79 C9 49 3D ... FF FF FF FF 04 A1 EF 26 55
//Host sends I2C_DATA_Len_reg.
AA 51 00 08
//Host reads data length.
0E 00 00 00
//Module responds to FW file frame. See Table 34: FW File Response Frame from Module.
AA 02 00 00 04 02 04 00 00 7F 87 1E 83 55

//Send firmware address (gnss_config.bin)
//Host sends the firmware address frame. See Table 21: Firmware Address Frame from Host.
AA 02 01 00 04 08 3D F0 00 8A CF 1E 9F 55
//Host sends I2C_DATA_Len_reg.
AA 51 00 08
//Host reads data length.
0E 00 00 00
//Host reads module's response to firmware address frame. See Table 22: Firmware Address Response Frame from Module.
AA 02 00 00 04 02 01 00 00 79 4C DC 68 55

//Firmware information (gnss_config.bin)
//Host sends the firmware information frame. See Table 23: Firmware Information Frame from Host.
AA 02 02 00 10 00 00 10 00 3B C6 EC D9 08 3F F0 00 01 00 00 00 59 71 90 37 55
//Host sends I2C_DATA_Len_reg.
AA 51 00 08
//Host reads data length.
```

0E 00 00 00

//Host reads module's response to firmware information frame. See [Table 25: Firmware Information Response Frame from Module](#).

AA 02 00 00 04 02 02 00 00 7B 0A 62 31 55

//Send start firmware upgrade command

//Host sends the start firmware upgrade frame. See [Table 26: Start Firmware Upgrade Frame from Host](#).

AA 02 03 00 00 89 0B A9 CE 55

//Host sends I2C_DATA_Len_reg.

AA 51 00 08

//Host reads data length.

0E 00 00 00

//Host reads module's response to start firmware upgrade frame. See [Table 27: Start Firmware Upgrade Response Frame from Module](#).

AA 02 00 00 04 02 03 00 00 7A C8 08 06 55

//Format flash (gnss_config.bin).

//Host sends the format flash frame. See [Table 28: Format Flash Frame from Host](#).

AA 02 21 00 08 08 3F F0 00 00 00 10 00 B9 BA 6F B9 55

//Host sends I2C_DATA_Len_reg.

AA 51 00 08

//Host reads data length.

0F 00 00 00

//Host reads module's format progress. See [Table 30: Format Progress Frame from Module](#).

AA 02 00 00 05 02 21 64 00 00 B5 A2 55 5C 55

//Host sends the format progress response frame. See [Table 31: Format Progress Response Frame from Host](#).

AA 02 21 00 05 00 00 64 00 00 E2 30 AC AF 55

//Host sends I2C_DATA_Len_reg.

AA 51 00 08

//Host reads data length.

0E 00 00 00

//Host reads module's response to format flash frame. See [Table 32: Format Flash Response Frame from Module](#).

AA 02 00 00 04 02 21 00 00 41 01 9A 88 55

//Send FW file to module (gnss_config.bin)

//Host sends the FW file frame. See [Table 33: FW File Frame from Host](#). (The frame size is 256 bytes.)

AA 02 04 10 04 00 00 00 00 01 01 01 ... FF FF FF FF B2 24 8D 72 55

//Host sends I2C_DATA_Len_reg.

AA 51 00 08

//Host reads data length.

0E 00 00 00

//Module responds to FW file frame. See [Table 34: FW File Response Frame from Module](#).

AA 02 00 00 04 02 04 00 00 7F 87 1E 83 55

4 Upgrade Example by QGNSS

This chapter provides a step-by-step guide to upgrading firmware via QGNSS.

4.1. Procedure in Download Mode for General Applications

Quectel GNSS MODULE EVB (V1.3) and the LC29H (AA) module are used as an example in this chapter. For details about the EVB, see [document \[5\] EVB user guide](#).

As shown in the figure below, connect the EVB to your PC via the GNSS-USB interface:

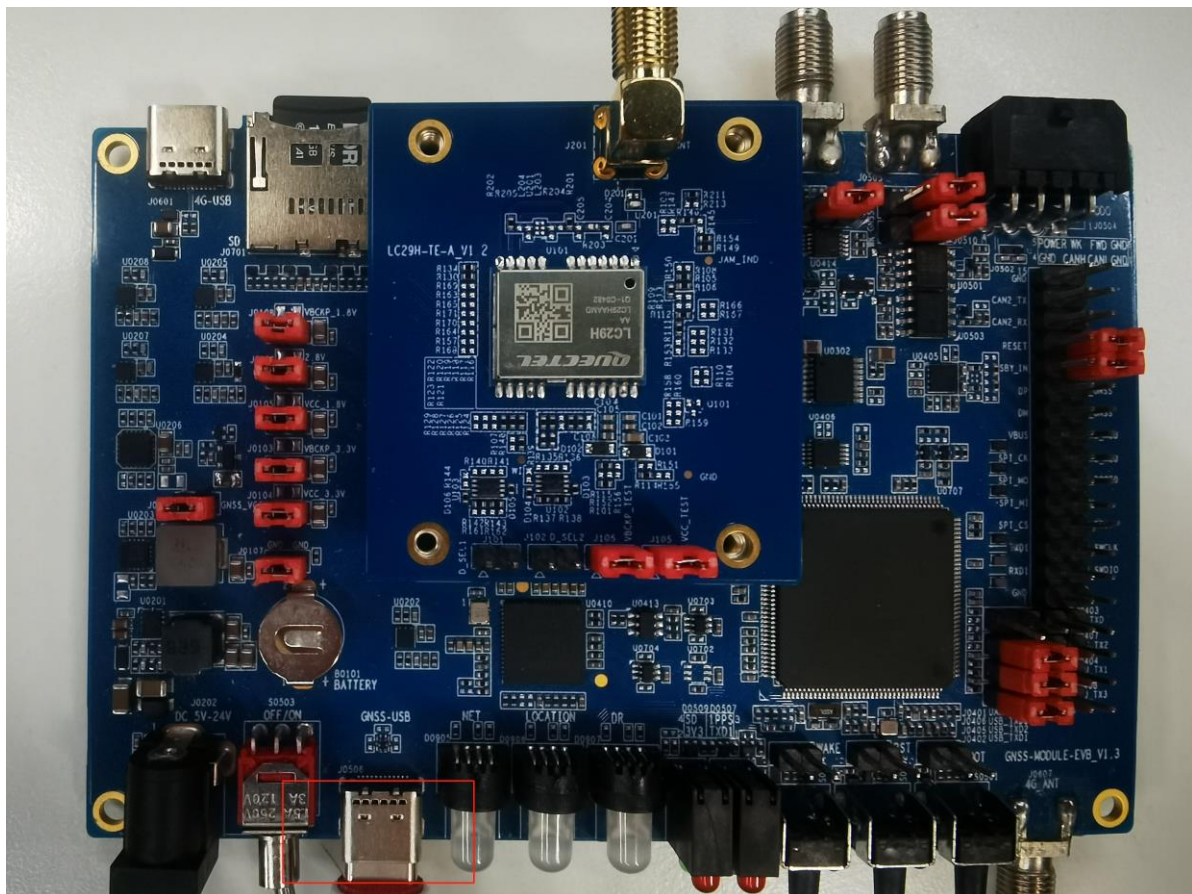


Figure 30: EVB Connection

Tips:

1. Before initiating the firmware upgrade, ensure that the module is connected to the PC and powered on normally. If it is properly connected and powered on, 4 random but consecutive COM port numbers appear in the “**Device Manager**”. Choose the lowest COM port number for communication with the GNSS module. See [Figure 31: Device Manager](#).
2. For this procedure, it is imperative to utilize QGNSS V1.9 or higher versions.

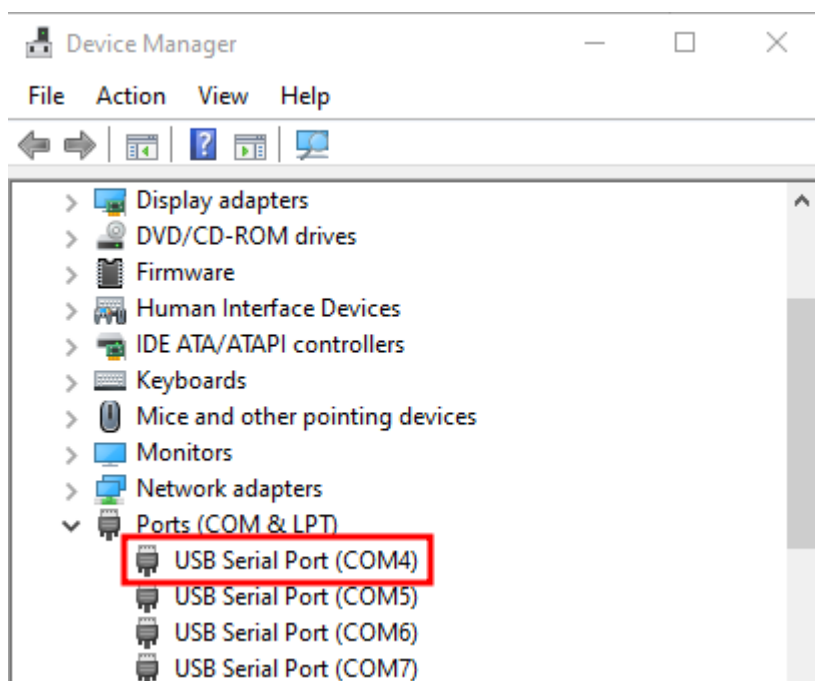


Figure 31: Device Manager

Firmware upgrade procedure steps are as follows:

Step 1: Open QGNSS tool and click  to access the “**Device Information**” window as shown in [Figure 32: Device Information](#).

Step 2: Choose the module to be upgraded.

Step 3: Choose port number.

Step 4: Choose baud rate for communication. Default baud rate: 115200 bps.

Step 5: Click “**OK**”.

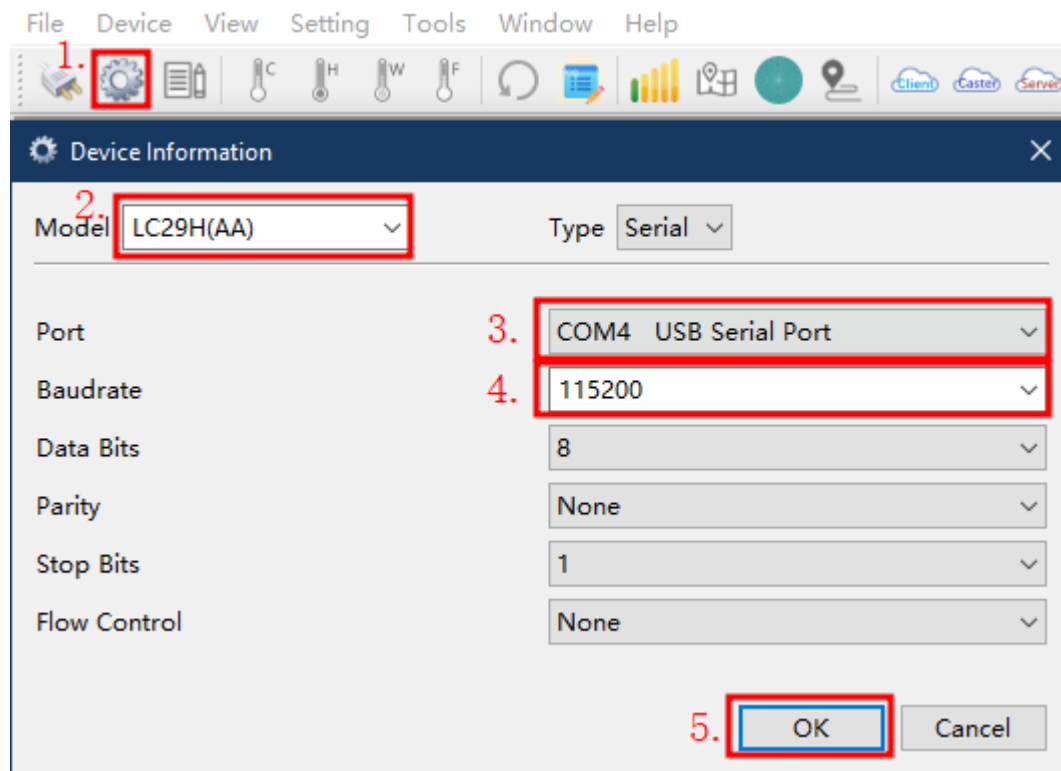


Figure 32: Device Information

Step 6: Click “Tools” as shown in [Figure 33: Firmware Download](#).

Step 7: Choose “Firmware Download” in the drop-down menu of “Tools”, as shown in [Figure 33: Firmware Download](#).

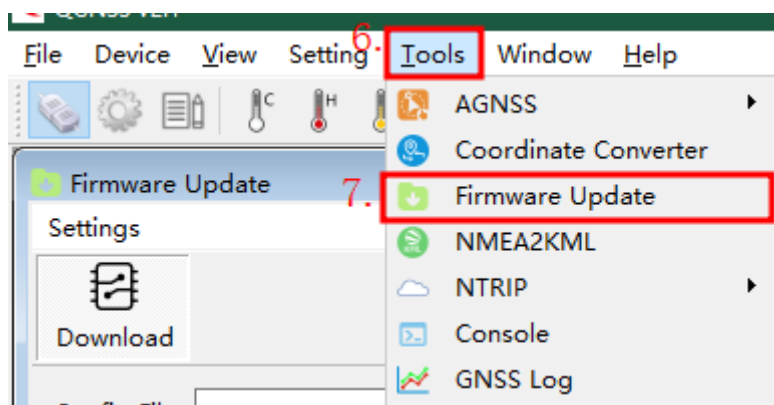


Figure 33: Firmware Download

Step 8: In the newly opened window, click “Settings” to select the baud rate for downloading firmware,

and then click  Download.

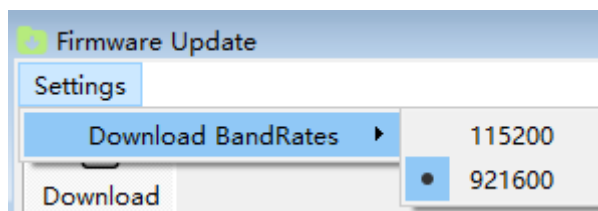


Figure 34: Select Download Baud Rate

Step 9: Click  to choose the firmware upgrade package. And then choose the .cfg file within the firmware upgrade package as shown in the [Figure 36: Firmware Selection](#).

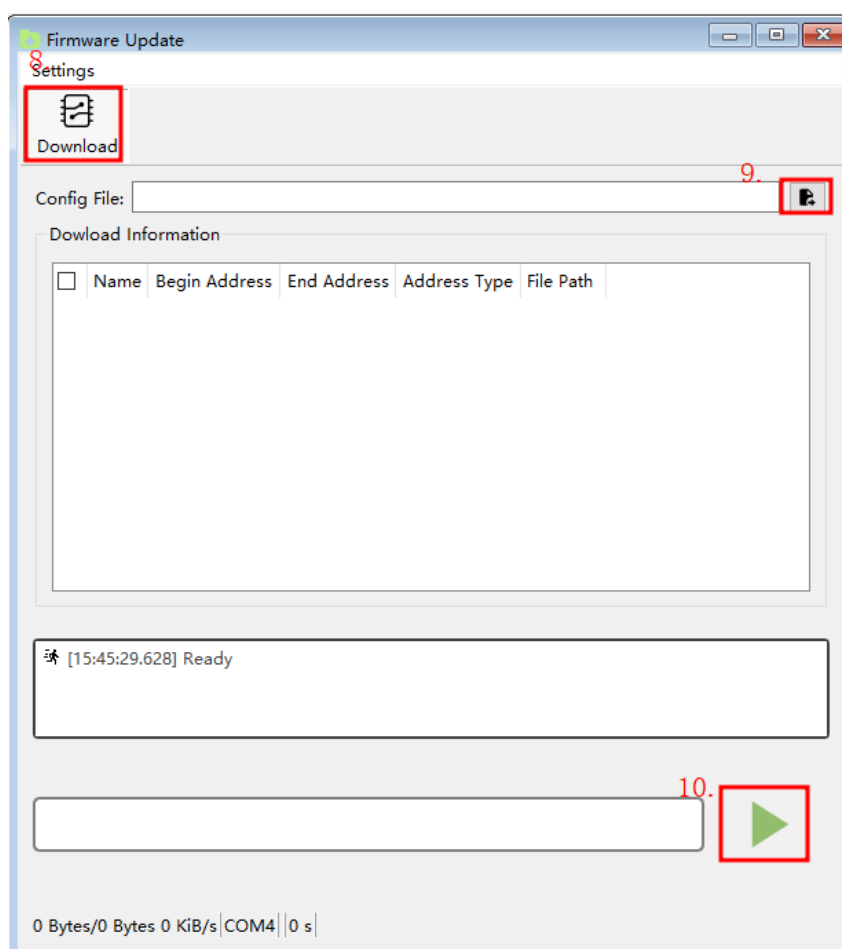


Figure 35: Load Firmware



Figure 36: Firmware Selection

Step 10: Click  to initiate the firmware upgrade process and wait for HANDSHAKE between the module and the PC as shown in [Figure 37: Wait for Handshake](#). Reset module within 20 s. Otherwise, the firmware upgrade will fail. In this case, you can retry from **Step 10**.

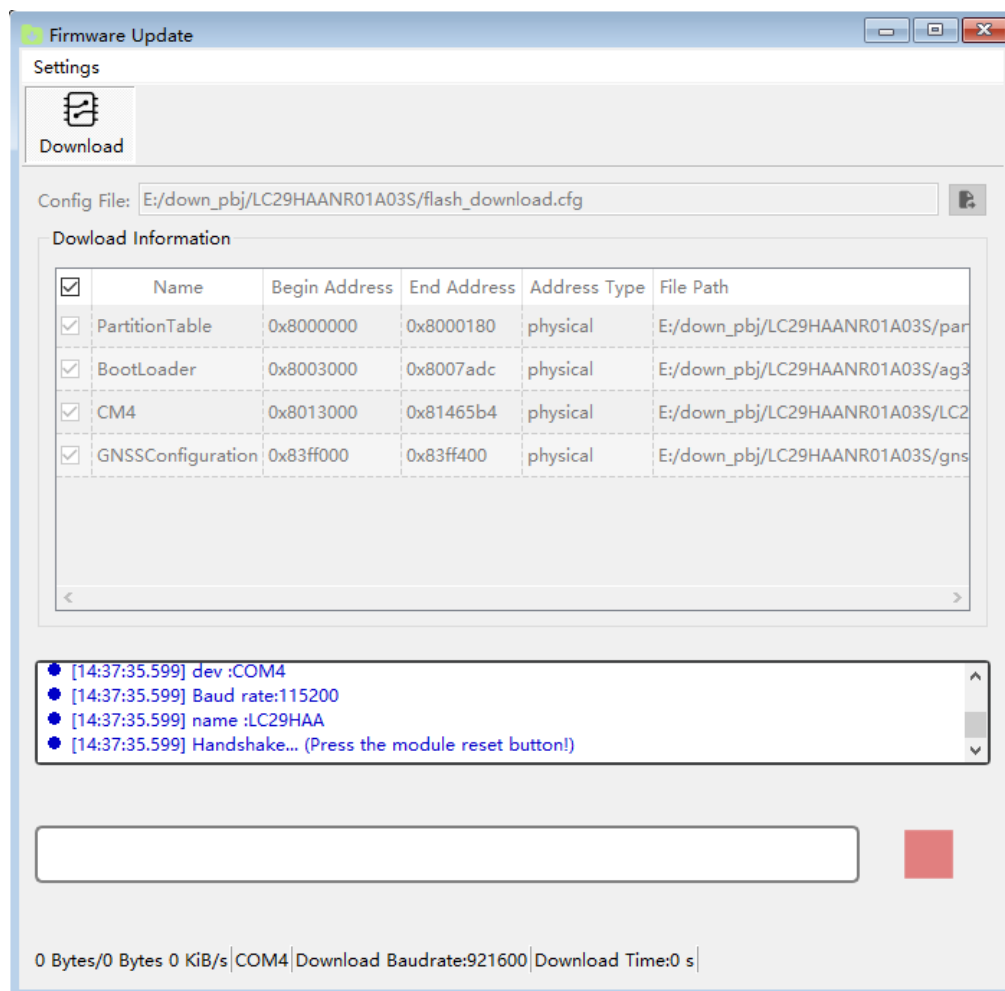


Figure 37: Wait for Handshake

Step 11: After a successful HANDSHAKE, the module waits for firmware upgrade.

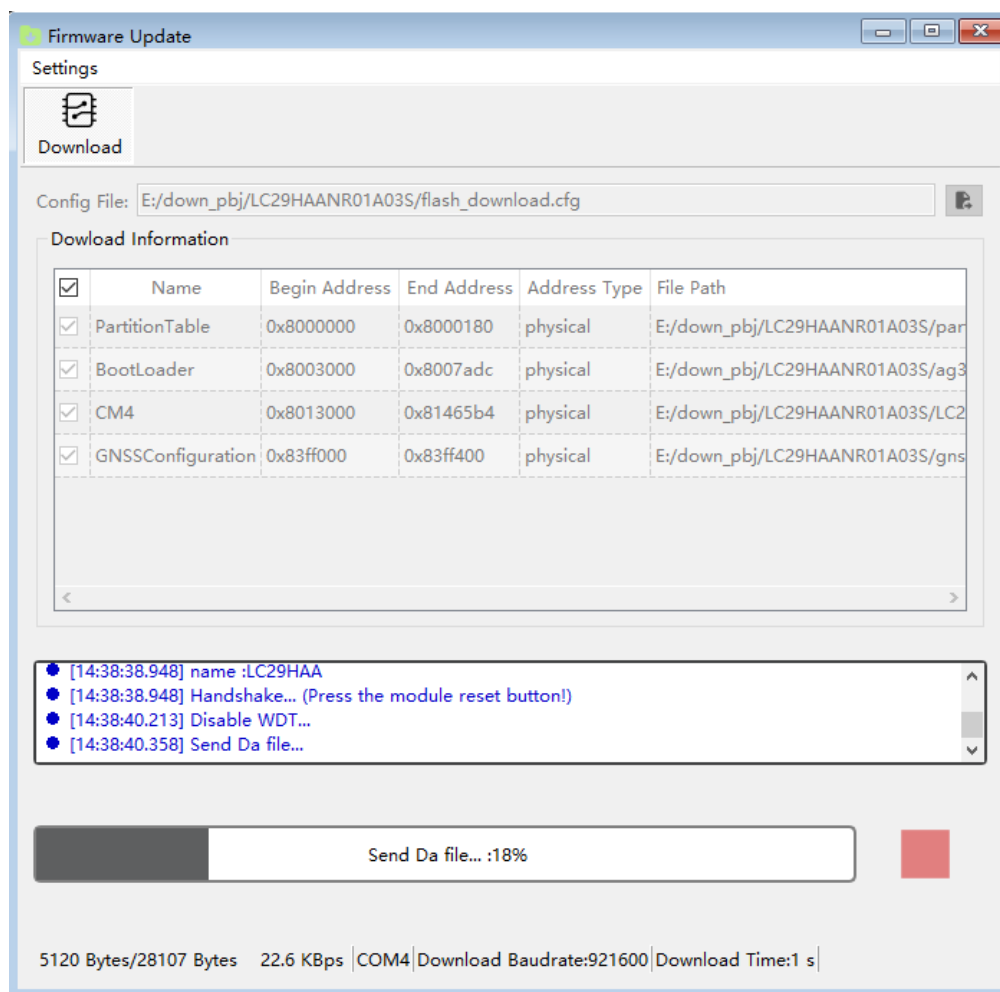


Figure 38: Handshake Succeeded

Step 12: Wait for upgrading to complete.

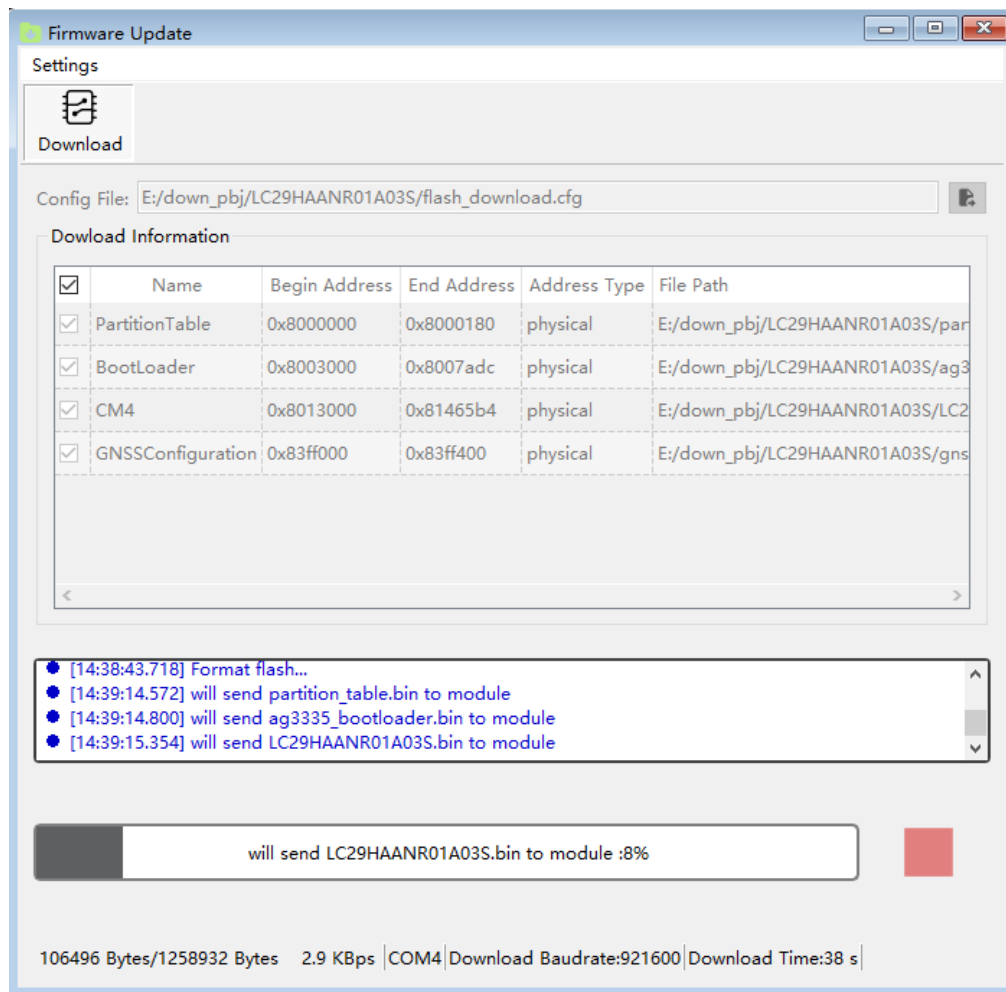


Figure 39: Wait for Upgrading to Complete

Step 13: Firmware upgrade is successful.

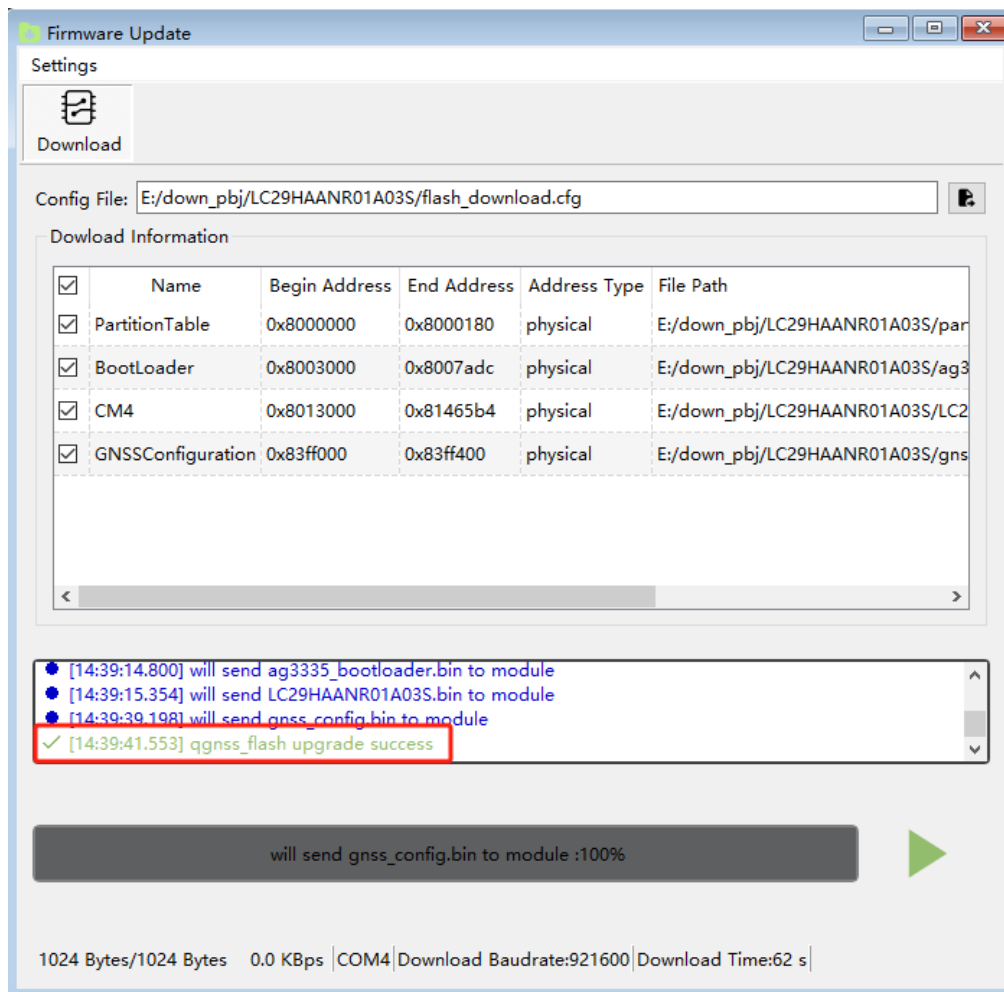


Figure 40: Firmware Upgrade Completed

Step 14: Reset the module to run on the new firmware.

4.2. Procedure in Download Mode for Moving Base Application

When upgrading the firmware of LC29H (EA) for the Moving Base application, the steps are almost identical to those outlined in [Chapter 4.1 Procedure in Download Mode for General Applications](#) except for the .cfg file to be selected.

In the Moving Base application, select the .cfg file with the “ms” mark in the filename as shown in [Figure 41: Firmware Selection of LC29H \(EA\) in Moving Base](#) to perform the firmware upgrade.

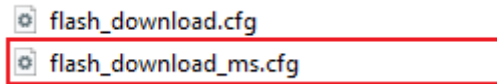


Figure 41: Firmware Selection of LC29H (EA) in Moving Base Application

After the firmware upgrade is complete, upon the first startup, the master module takes approximately 80 s to upgrade the slave module. Wait for the master module to output normally (as shown in [Figure 42: Upgrade Completed](#)) before using them.

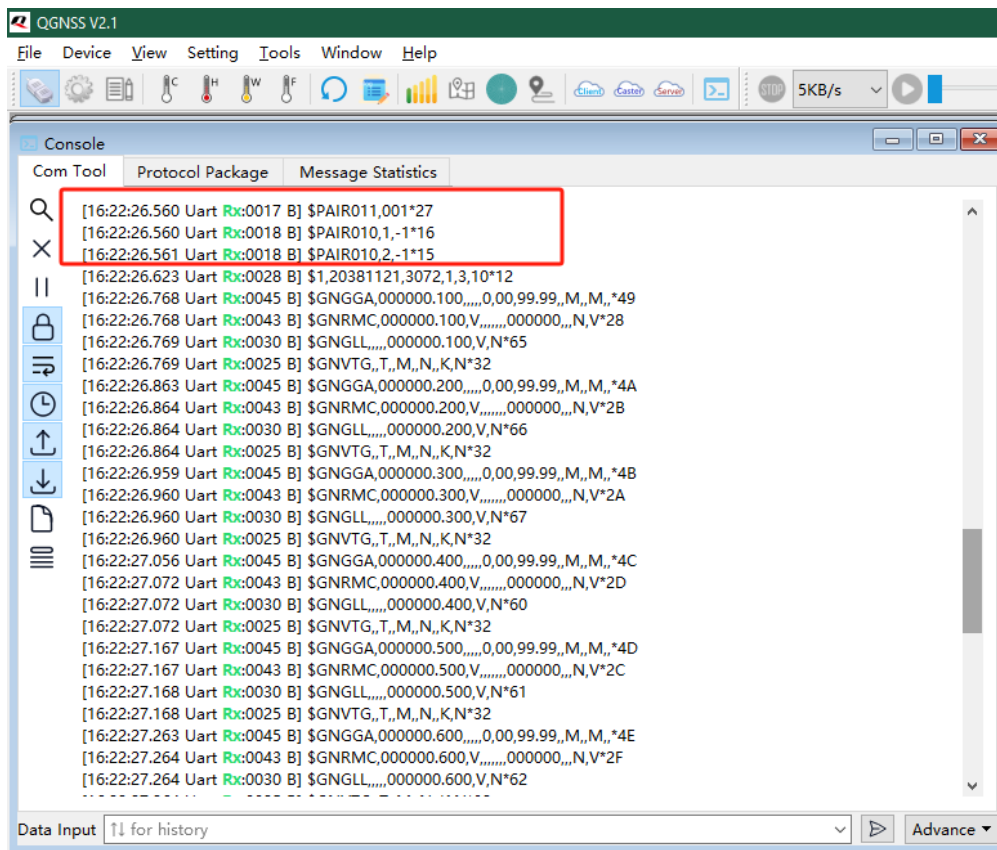


Figure 42: Upgrade Completed

5 Appendix References

Table 35: Related Documents

Document Name
[1] Quectel L89(HA) GNSS Protocol Specification
[2] Quectel LC29H&LC79H Series GNSS Protocol Specification
[3] Quectel L89(HD)&LC29H(AI) GNSS Protocol Specification
[4] Quectel LC29H Series SPI Application Note
[5] Quectel GNSS-MODULE EVB User Guide

Table 36: Terms and Abbreviations

Abbreviation	Description
BROM	Boot ROM
CRC	Cyclic Redundancy Check
DA	Download Agent
FW	Firmware
GNSS	Global Navigation Satellite System
I2C	Inter-Integrated Circuit
SPI	Serial Peripheral Interface
UART	Universal Asynchronous Receiver/Transmitter
WDT	Watchdog Timer